A Local RAG Retrieval Benchmarking Pipeline for Corpus-Specific Embedding-Model Selection

Murat Karakaya[1*]

[1*]Department of Software Engineering, TED University, Ankara, Türkiye.

Corresponding author(s). E-mail(s): murat.karakaya@tedu.edu.tr;

Abstract

Practitioners who operate retrieval-augmented generation (RAG) systems often manage several heterogeneous local corpora, and an embedding model that is suitable for one collection should not be transferred automatically to another. This paper presents a configurable local benchmarking pipeline for corpus-specific embedding-model selection. The completed workflow processes each corpus separately through document preparation, synthetic query generation, relevance-judgment construction, multi-model retrieval evaluation, and automated report generation. Each dataset-level report combines retrieval metrics, embedding/insertion throughput, query latency, and preserved artifacts; aggregate reporting is used only as secondary cross-corpus context.

The pipeline is demonstrated through one frozen three-corpus experimental instantiation (education_eng, education_tr, law_tr) with five sentence-transformer embedding candidates denoted M1–M5, 60 source documents, 1,914 chunks, 150 generated synthetic queries, and 9,296 judged query–chunk pairs, including 952 positive relevance judgments. The purpose of the demonstration is not to make these particular model rankings the main contribution, but to show that the same controlled workflow can expose substantial corpus- and priority-dependent differences across retrieval quality and operational metrics. In the snapshot, the dataset-level reports point to M1 for education_eng when MAP@10 and latency are both considered, to M5 as the quality-first candidate for the two Turkish corpora, and to M3 as a lower-latency Turkish compromise. These different readings illustrate the practical need for a pipeline that lets practitioners evaluate candidate embeddings on their own datasets and application requirements instead of assuming one transferable winner. Each report also carries query-level bootstrap intervals over the 148 scored positive-bearing queries, so a reader can see which parts of a corpus ranking the benchmark actually resolves. All evidence remains conditional on the reported snapshot: synthetic queries, unvalidated LLM qrels, a fixed retrieval geometry, a five-model roster, and single-run timing.

Keywords: Local RAG benchmarking, Embedding-model selection, Corpus-specific evaluation, Retrieval evaluation, Synthetic queries, Relevance judgments

1 Introduction

Retrieval-augmented generation (RAG) systems depend on retrieval quality to ground generated answers, so embedding-model choice directly shapes the evidence that a generator can use. In local deployments, that choice is usually made for document collections controlled by a practitioner rather than for the generic benchmark corpora used in public rankings. One organization may maintain several corpora that differ in language, domain vocabulary, document structure, source provenance, scale, and information needs. A model that is suitable for one of those collections should therefore not be transferred automatically to another collection merely because it performs well on a public leaderboard or on a previous local run (Lewis et al. 2020).

Generic leaderboards and informal trials remain useful starting points, but neither determines every local choice. Public rankings summarize behavior on shared tasks rather than on the practitioner's own documents, while ad hoc trials can change multiple protocol variables at once, including preprocessing, chunking, query construction, relevance judgments, retrieval geometry, candidate roster, and reporting criteria. Manual comparison also becomes costly when the same practitioner needs to evaluate several local corpora or corpus subsets under comparable conditions. The practical need is a controlled workflow that applies the same evaluation protocol separately to each corpus and produces corpus-specific evidence before any cross-corpus summary is considered.

This paper addresses that model-selection problem with a reusable local benchmarking pipeline for embedding-model evaluation under corpus-specific RAG requirements. The central argument is that practitioners should not assume that a single embedding model will be equally suitable for every document collection under their control. Rather than treating three worked demonstrations as proof of a universal optimum or its absence, the paper shows how each collection can be benchmarked independently with the same candidate roster, fixed retrieval protocol, and dataset-level report. The resulting report is the primary model-selection artifact for a given corpus and application setting, while aggregate reporting provides secondary context for comparing cross-corpus behavior.

The pipeline is demonstrated through a fixed experimental snapshot covering three document collections and five embedding models. The snapshot contains 60 source documents, 1,914 chunks, 150 synthetic queries, and 952 positive query–chunk relevance judgments (qrels, the ground-truth labels that mark which chunks are relevant to each query). The English and Turkish education corpora and the Turkish law/regulation corpus provide heterogeneous worked cases for examining language- and domain-associated ranking differences or stability under one shared protocol. These corpus characteristics are important for partitioning and interpreting local benchmark runs,

but the snapshot is not a causal language or domain study. Setup details, including provenance, fixed retrieval geometry, query accounting, and measured operational quantities, are provided in Section 4; comparative evidence is reported in Section 5.

Contributions and Research Questions

The paper makes four contributions to local RAG benchmarking practice. First, it defines a repeatable per-corpus benchmark-construction workflow that turns a local document collection into reusable corpus, query, qrels, ranking, timing, and report artifacts, taking an unlabeled document collection as input rather than presupposing the labeled benchmark that existing evaluation libraries require. Second, it provides shared-protocol candidate comparison by evaluating the same embedding-model roster separately on each corpus under fixed retrieval and reporting settings. Third, it produces dataset-level reports and aggregate summaries that combine retrieval quality with the reported operational behavior: embedding/insertion throughput and query latency. Fourth, it preserves benchmark artifacts for inspection and report regeneration so that the evidence behind each corpus-specific shortlist can be revisited.

Research Questions

Guided by these contributions, the empirical analysis addresses four broad research questions:

- RQ1 (Pipeline utility): To what extent does the pipeline produce decision-support evidence—a ranked model shortlist with retrieval-quality metrics, operational-cost measurements, and query-level uncertainty estimates—for embedding-model selection on each local document collection?
- RQ2 (Cross-language shortlist stability): Under one fixed benchmark protocol, to what extent do the corpus-specific shortlists change between an English and a Turkish corpus of the same domain, and how far does the query-level uncertainty layer support any change that is observed?
- RQ3 (Cross-domain shortlist stability): Under one fixed benchmark protocol, to what extent do the corpus-specific shortlists change or remain stable between two Turkish corpora of different domains?
- RQ4 (Operational decision support): How does the pipeline combine retrieval effectiveness, operational cost, and query-level uncertainty into fit-for-purpose deployment guidance?

Pipeline utility in RQ1 means the ability to define and run controlled configurations, preserve the resulting artifacts, and compare corpus-specific outcomes. RQ2 is answered through a bounded comparison of the English and Turkish education corpora, not through a causal language experiment. RQ3 is answered through the Turkish education and Turkish law/regulation corpora, where stability is as informative as difference because the question concerns whether a separate corpus run changes or preserves the shortlist. For RQ4, *operational cost* refers only to the quantities reported in this paper: combined corpus embedding and index-insertion throughput, and combined query-embedding and vector-search latency. Final deployment decisions may

still require practitioner review, human validation, or additional parameter-sensitivity runs.

The pipeline is implemented as a configurable workflow rather than as a single fixed experiment. Its configuration file allows a practitioner to change the dataset, candidate embedding-model roster, metric flags and cutoff depth, chunk size and overlap, embedding batch size, collection reinitialization policy, ChromaDB distance space, number of generated queries, candidate-pool depth for qrels construction, pooling strategy, LLM judge settings, and aggregate-reporting scope. The empirical snapshot reported in this paper freezes one documented configuration so that the three corpus-level demonstrations are comparable; therefore, the reported rankings are conditional on that configuration, synthetic queries, LLM-generated relevance judgments, the five evaluated models, non-normalized squared-Euclidean retrieval, and single-machine timing.

A preliminary version of this work was presented at the International Conference on Computer Science and Intelligence Systems (Karakaya 2025), where the workflow was demonstrated on a single English technical corpus to establish the feasibility of automated local embedding-model benchmarking. The present paper extends that work in five respects: it evaluates the same candidate roster separately across three language- and domain-differentiated corpora with explicit source-provenance boundaries; it constructs model-independent pooled relevance judgments within the evaluated roster; it reports query-level bootstrap uncertainty for every corpus ranking; it treats the dataset-level report rather than any aggregate summary as the primary selection artifact; and it combines retrieval quality with measured operational cost for corpus-specific shortlisting. Section 2.6 states this relationship in full.

The remainder of the paper is organized as follows. Section 2 positions the paper against prior retrieval-evaluation and RAG-benchmarking work, Section 3 describes the proposed workflow, Section 4 documents the selected experimental instantiation, Section 5 reports corpus-specific and aggregate evidence, Section 6 discusses practitioner implications and transfer limits, and Section 7 concludes with implications and future directions.

2 Related Work

This paper is a practitioner-facing benchmark-*method* contribution, so we organize prior work around four design questions that determine whether a local embedding benchmark can support corpus-specific model shortlisting: how retrieval is evaluated in public and local settings, how synthetic queries and relevance judgments are constructed and judged, how embedding rankings vary with language- and domain-associated corpus characteristics, and how reproducibility and benchmark governance are controlled. We then relate the proposed pipeline to the general-purpose evaluation tooling a practitioner could otherwise assemble, and state the gaps these threads leave open for the present study.

2.1 Local, Practitioner-Facing RAG Retrieval Evaluation

RAG systems depend on retrieval quality to maintain answer grounding and downstream reliability, so retrieval design remains a central determinant of end-task behavior. Foundational studies show that weaker retrieval can directly reduce grounding quality, and survey evidence frames IR–LLM integration as a multi-component pipeline spanning query rewriting, retrieval, reranking, and reader/generator interaction, while highlighting hallucination risk under weak grounding (Lewis et al. 2020; Zhu et al. 2026). Public benchmark suites such as BEIR and MTEB provide broad shared-testbed evidence for retrieval and embedding models, but their strength as reusable leaderboards also limits their ability to answer a practitioner's local corpus question directly: whether the same candidate roster should be selected differently for separate corpora under the practitioner's control (Thakur et al. 2021; Muennighoff et al. 2023).

Pipeline-focused studies on local and enterprise collections move closer to deployment-oriented evaluation but remain heterogeneous in objectives and scoring. Examples include closed-domain health-information workflows, industrial semantic search, telecom retrieval architectures, domain-adaptive engineering retrieval, requirements-to-code traceability, and reviews that emphasize resource and runtime costs (Cummings et al. 2025; Yozkan and Supriyanto 2025; Khan et al. 2025; Szabó et al. 2025; Ali et al. 2024; Yunianto et al. 2020). These works are useful evidence that local context matters, but they usually optimize one application pipeline rather than offering a repeatable shared-protocol comparison of a candidate embedding roster across the practitioner's own corpora.

2.2 Synthetic Queries, LLM-as-Judge, and Qrels Construction

Because fully curated relevance labels are expensive, recent work uses language models to generate queries and to judge relevance. Agentic and multimodal RAG settings embed an LLM-as-a-judge that critiques responses across quality and safety dimensions, and enterprise deployments report LLM judging as useful but still an imperfect substitute for human assessment; multilingual judge calibration is itself non-trivial, with calibrated, uncertainty-aware scoring improving human alignment (Bellard et al. 2025; Wang and Birman 2025; Kunuku and Dehbozorgi 2025; Taemkaeo et al. 2026; Wen et al. 2026). Evaluation-design studies make the reliability question explicit by comparing LLM and human judgments under controlled protocols: agreement is substantial but condition-dependent, and judge configuration should be treated as a measurable design component rather than a fixed shortcut (Thakur et al. 2025; Abdallah et al. 2025; Miyaji et al. 2025). Work on synthetic-data construction adds a further reference point for qrels design, in that synthetic query labeling can be competitive with real-data training (Bassani and Pasi 2021).

A specific design choice within this thread is how the *candidate pool* that a judge labels is constructed. In TREC-style information-retrieval evaluation, relevance judgments are commonly pooled from the outputs of multiple retrieval systems so that the judged set does not privilege any single retriever by construction (Voorhees 2002). The RAG-evaluation studies above similarly treat judgment collection across retrieval

stages and judge configuration as first-class design variables (Thakur et al. 2025; Abdallah et al. 2025; Miyaji et al. 2025). When a benchmark instead draws its candidate pool from one retriever that is also under evaluation, relevant passages that only other systems surface can be missed and silently counted as non-relevant, biasing the comparison toward the pool-generating model (Zobel 1998; Buckley and Voorhees 2004). The present paper adopts model-independent pooling within the evaluated roster for this reason.

2.3 Cross-Language and Cross-Domain Embedding Selection

Benchmark-oriented studies show that embedding rankings can depend on corpus characteristics and language context: corpus type and query format explain retrieval variance at a level comparable to model choice and shift rankings across corpora, and domain-specific question-answering rankings can diverge from public benchmarks while exposing practical efficiency tradeoffs (Mikkelsen 2026; Antal 2025). Controlled local-corpus studies add complementary evidence from enterprise retrieval, domain-adaptive embeddings, embedding migration, and corpus-dependent similarity analyses (Cohen et al. 2025; Luo et al. 2026; Yoon and Arik 2025; Dvořáčková 2025). Domain- and language-specialized studies reinforce that protocols tuned for one setting do not transfer automatically: task-aligned embeddings help environmental question-answering, and multilingual, Turkish-focused (TR-MTEB), and Persian benchmarks show ranking behavior conditioned on similarity formulation and corpus composition, alongside an open-source embedding survey weighing dimensionality, latency, and memory tradeoffs (Sajja et al. 2025; Baysan and Gungor 2025; Kalantari et al. 2024; Abimbola et al. 2026). Further multilingual studies make transfer sensitivity explicit through translation-based embedding, two-phase cross-lingual retrieval, language-specific models, and evidence that cross-lingual transfer can still degrade for low-resource settings (Iscan et al. 2024; Bagheri Nezhad and Agrawal 2025; Mekonnen et al. 2025; Guţu and Popescu 2025; Iana et al. 2026). This body of work supports the practical premise that language and domain should guide corpus partitioning and interpretation, but it does not remove the need to benchmark each local corpus under the same candidate roster and protocol.

2.4 Reproducibility and Benchmark Governance

A final thread emphasizes that evaluation quality depends on operational controls, not only model choice. ReproRAG quantifies retrieval reproducibility under changes in embedding models, precision, retrieval algorithms, and hardware; an end-to-end storage-aware framework profiles system-level benchmarking; RAGBench and the TRACe framework support explainable large-scale evaluation while reporting that LLM-based evaluators can underperform a fine-tuned RoBERTa evaluator; and RAGOps studies extend the lifecycle and indexing-choice perspective (Wang et al. 2025; Xu 2025; Friel et al. 2024; Xu et al. 2025). These works motivate explicit reproducibility conditions and preserved raw artifacts, yet reusable artifacts for repeatable *local* per-corpus embedding selection are seldom the primary deliverable.

2.5 Relation to General-Purpose Evaluation Tooling

The threads above motivate building a local benchmark; this subsection states what the proposed pipeline adds relative to the general-purpose tooling a practitioner could otherwise assemble, and where it depends on that tooling rather than replacing it. The comparison concerns documented scope and intended use, not a measured head-to-head evaluation. Four families are relevant. Shared testbeds such as BEIR, MTEB, and its multilingual successor supply corpus, queries, and relevance judgments together, so they answer which model performs best on a fixed set of public tasks; their evaluation code can be pointed at a local collection, but only once that collection is already accompanied by a query file and a qrels file, and unjudged pairs are then treated as non-relevant by convention (Thakur et al. 2021; Muennighoff et al. 2023; Enevoldsen et al. 2025). Evaluation and dataset libraries such as `ir_datasets`, `ir_measures`, and the Sentence-Transformers `InformationRetrievalEvaluator` standardize access to existing collections and compute ranking metrics from queries, judgments, and runs, and reproducible retrieval toolkits such as Pyserini execute retrievers over such resources under controlled conditions; each of these consumes a labeled benchmark rather than producing one (MacAvaney et al. 2021, 2022; Reimers and Gurevych 2019; Lin et al. 2021). LLM-based assessment components such as UMBRELA supply a relevance judge, but not the surrounding corpus-to-report workflow (Upadhyay et al. 2024). Finally, RAG evaluation frameworks such as RAGAS, ARES, and RAGBench score an assembled generation pipeline on dimensions such as context relevance, answer faithfulness, and answer relevance; RAGAS additionally synthesizes evaluation data from supplied documents, but the resulting items pair a question with the contexts that produced it, which serves answer-quality measurement rather than ranked-retrieval comparison over the full chunk set (Es et al. 2024; Saad-Falcon et al. 2024; Friel et al. 2024). Application frameworks such as LangChain and LlamaIndex expose evaluation utilities of the same kind inside an application, again expecting a labeled dataset supplied by the user (Chase 2022; Liu 2022).

Against this background, the contribution of the present paper is a composition rather than any single stage. Neither corpus chunking, synthetic query generation, LLM-assisted judging, pooling, nor the reported ranking metrics is new here, and the metric layer is deliberately delegated to an existing implementation (Reimers and Gurevych 2019). What the pipeline adds is that it takes an unlabeled local document collection as its input and produces a per-corpus model-selection report as its output, and that it fixes four choices in between which the general-purpose tools leave to the practitioner: the candidate pool submitted for judging is formed model-independently across the entire evaluated roster rather than from a single retriever; negative judgments are retained rather than discarded, so that judged-non-relevant and unjudged pairs remain distinguishable; one frozen protocol is applied to every candidate model over isolated per-model indexes; and the resulting report presents retrieval quality together with measured embedding, insertion, and query-time cost, because a local deployment decision depends on both. The pipeline is therefore positioned upstream of the answer-quality frameworks rather than in competition with them: a practitioner may use it to narrow the embedding roster for one corpus and then evaluate the assembled RAG system with those frameworks. Its outputs are written as a corpus, query,

and qrels triple in the layout such harnesses expect, so a benchmark constructed here can also be re-scored with external evaluation tools.

2.6 Gap Statement and Positioning

Across these threads, five gaps are central to this paper. Public benchmarks such as BEIR and MTEB are essential shared references, but they do not by themselves build a benchmark from a practitioner's own corpus. Many local RAG studies optimize one application pipeline rather than supporting practitioner-oriented per-corpus model shortlisting under one shared protocol (**RQ1**); language- and domain-associated corpus comparisons of the same evaluated roster are less often treated as local reporting outputs (**RQ2**, **RQ3**); descriptive quality–speed tradeoffs are reported less often than answer-quality gains (**RQ4**); reusable benchmark artifacts that make local evaluation inspectable and repeatable are seldom the deliverable; and general-purpose evaluation tooling either supplies a fixed public benchmark or presupposes the labeled benchmark that a local corpus does not come with (Section 2.5). The candidate-pool construction issue noted above is a further method-level gap that this paper treats explicitly. In response, this paper contributes a reusable, practitioner-facing pipeline that evaluates one embedding roster under a fixed protocol separately for each local corpus, constructs relevance judgments with model-independent pooling within that roster, and reports retrieval quality, operational cost, and query-level uncertainty as corpus-specific artifacts.

A preliminary version of this line of work demonstrated the feasibility of automated local embedding-model benchmarking on a single English technical corpus, using synthetic-query generation, LLM-assisted relevance construction, and multi-model retrieval scoring (Karakaya 2025). The present paper shifts the contribution from one-corpus feasibility to practitioner-facing, corpus-specific model selection. It evaluates the same candidate roster separately across three language- and domain-differentiated local corpora with explicit source-provenance boundaries, treats dataset-level reports as the primary selection artifact, adds model-independent pooled qrels within the evaluated roster, reports query-level uncertainty, and incorporates retrieval-quality and timing evidence for corpus-specific shortlisting (TED University 2026; Bankacilik Duzenleme ve Denetleme Kurumu (BDDK) 2026).

3 Method

This study proposes a reusable, configuration-driven local benchmarking pipeline for selecting embedding models under corpus-specific Retrieval-Augmented Generation (RAG) requirements. The central methodological assumption is that a practitioner should not assume one embedding model will be suitable for every document collection under their control. Instead, each corpus is routed through the same staged workflow separately, producing its own chunked corpus, synthetic queries, relevance judgments, model-specific vector indexes, retrieval metrics, timing summaries, and a dataset-level report. Within a benchmark comparison, the selected candidate roster and protocol are held constant so that the corpus-specific reports remain comparable, but the primary model-selection artifact remains the report for the corpus being deployed. When

heterogeneous corpus subsets are benchmarked separately, an aggregate report can provide secondary comparative context across topic- or language-specific partitions rather than a command to deploy one global model.

3.1 Pipeline Overview

The pipeline operates at the dataset level and begins with explicit dataset routing, as illustrated in Figure 1. Each corpus is processed, indexed, evaluated, and reported in isolation so that document inputs, chunked corpora, synthetic queries, qrels, vector collections, ranking artifacts, timing traces, and reports remain separated across datasets. This design reduces cross-dataset contamination and allows one practitioner to repeat the same methodological template for document collections that differ in language, domain, structure, or scale. After routing, the workflow advances through document ingestion, chunking, query generation, and relevance construction, thereby producing the benchmark artifacts required for comparative retrieval within that corpus.

Before a benchmark run, the practitioner selects the document collection and configures the candidate models and evaluation settings according to the intended comparison. Those choices define that experimental instantiation. Once the comparison begins, the selected values are held constant across candidate models and, when cross-corpus comparison is intended, across corpus runs. Thus, a *fixed protocol* in this method denotes within-comparison experimental control rather than hard-coded pipeline parameters; a different configuration can be evaluated through a separate controlled run.

Once these corpus-level artifacts have been created, the pipeline branches into model-specific indexing and evaluation. Separate vector collections are built for each candidate embedding model, and each model from the shared roster is evaluated against the same query and qrels resources for that corpus rather than against model-specific or manually curated evidence. Holding the selected configuration constant makes the resulting dataset-level reports comparable while preserving corpus-specific model selection as the primary objective. Retrieval evaluation is followed by an uncertainty stage that resamples the per-query metric values, so that each dataset report states not only how the candidates rank but which parts of that ranking the corpus actually resolves. The final stage consolidates the resulting outputs into dataset-level reports and, when multiple corpora are available, an optional aggregate comparison report. The sequence of stages and their intermediate products are summarized in Table 1 and illustrated conceptually in Figure 1.

3.2 Corpus Preparation

The first stage converts each local document collection into a benchmarkable text corpus. Source documents are ingested from the local corpus and parsed into textual segments; in the present experimental snapshot, these source documents are provided as PDF files. To preserve local context while keeping units manageable for downstream embedding models, the extracted text is segmented by recursive character-based chunking with controlled overlap between adjacent segments. This strategy reduces

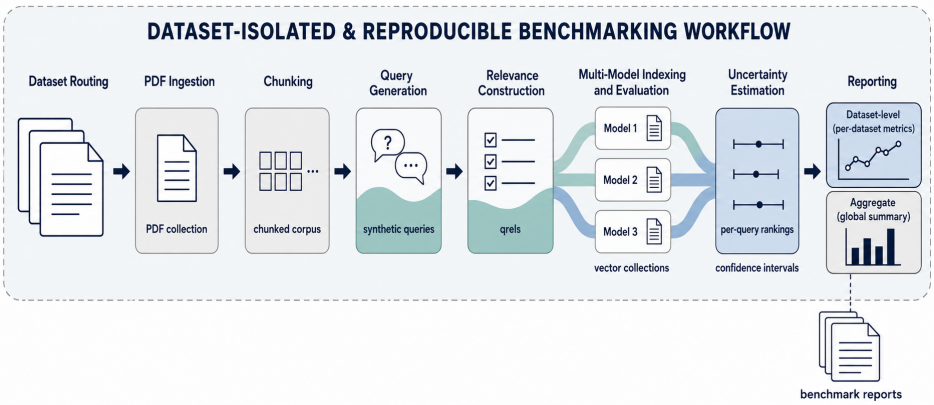


Fig. 1 Conceptual overview of the proposed embedding benchmarking pipeline. The figure summarizes the flow from a local document collection to corpus-specific reporting; Table 1 gives the authoritative stage sequence, including the uncertainty-estimation stage of Section 3.6. The three model boxes are schematic: the pipeline indexes and evaluates a roster of n candidates, and the reported snapshot uses five.

the probability that semantically dependent passages are split into entirely disconnected units, while still producing retrieval items that can be embedded and indexed efficiently. The fixed parameterization used in the experimental instantiation of this workflow is summarized in Section 4.4.

Dataset-level isolation is maintained during corpus preparation. Each dataset yields its own chunked corpus and retains its own storage path for downstream artifacts. As a result, the same pipeline can be reused across multiple corpora without mixing documents, collections, queries, qrels, indexes, evaluations, or reports between experiments.

3.3 Synthetic Query and Relevance Construction

After corpus construction, the pipeline builds benchmark queries and relevance judgments through the sequence shown in Figure 2. The procedure assigns the language model two distinct roles that we keep terminologically separate throughout the paper: a *query generator* that turns a seed passage into an information need, and a *relevance judge* that labels candidate passages against that need. The two roles are separated deliberately because their validity concerns differ—query realism on one side, judgment reliability on the other—and should be examined independently.

For each dataset, construction proceeds in five steps:

1. **Seed sampling.** Sample seed chunks from the chunked corpus under a fixed random seed, one seed chunk per query and without replacement, so that each chunk seeds at most one query.
2. **Query generation.** For each seed chunk, prompt the query generator to produce one concise natural-language query in the same language as the seed. This anchors queries to the target corpus while avoiding the cost of manually authoring a query set for each new dataset.

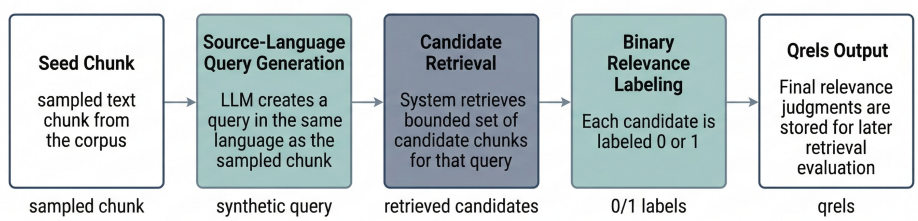


Fig. 2 Workflow for synthetic query generation and binary relevance construction.

3. **Pooled candidate retrieval.** For each query, form a candidate pool by taking the top-ranked chunks retrieved by *every* evaluated embedding model and unioning them after deduplication. Pooling across all models, rather than trusting a single retriever, reduces the privilege given to any one evaluated model when constructing the candidate set.
4. **Relevance judging.** Present each query–candidate pair to the relevance judge, which returns a binary relevant/non-relevant label under a fixed instruction.
5. **Qrels assembly.** Store the judgments as a qrels artifact that records *both* positive and negative labels, so that non-relevant decisions remain auditable rather than discarded, and freeze the query set and qrels before any embedding model is scored so that every model is evaluated against the same relevance resource.

The concrete candidate-pool size, query count, and seed policy used in this paper are reported in Section 4.4. Model-independent pooling is a deliberate benchmark-design choice: as discussed in Section 2, constructing qrels from a single retriever's candidate list can bias the evaluation toward that retriever. This pooling step bounds single-model bias within the evaluated roster, but it does not guarantee complete ground truth; relevant chunks missed by all pooled retrievers can remain unjudged, a known limitation of pooled evaluation (Zobel 1998; Buckley and Voorhees 2004).

The query-generation and LLM-assisted relevance-construction stages build on the preliminary workflow in Karakaya (2025), but they are used here inside a broader per-corpus selection pipeline with model-independent pooling and dataset-level reporting.

3.4 Embedding Index Construction

The comparative evaluation stage uses a fixed set of sentence-transformer models spanning lightweight, higher-capacity, and multilingual alternatives within a single benchmarking design. Each model is assigned its own vector collection so that embedding generation, storage, and retrieval remain isolated across models. This separation is methodologically important because it ensures that observed differences arise from the embedding models themselves rather than from shared or overwritten index state. The exact benchmarked model roster is specified in the experimental setup (Sentence Transformers 2026a).

Model-specific collections are built from the same chunked corpus. This means that corpus content, query set, and relevance judgments remain fixed while only the embedding function changes across runs. Such control is necessary if the benchmark

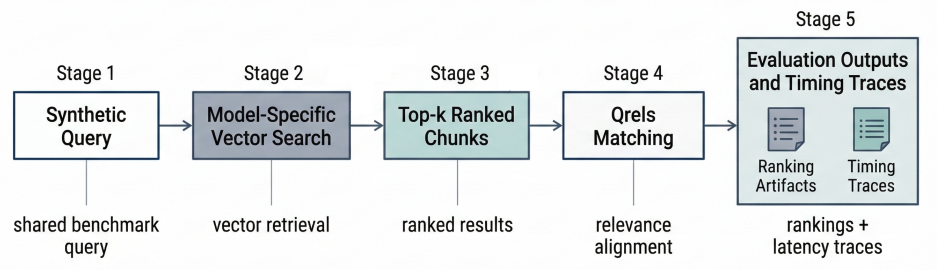


Fig. 3 Retrieval evaluation workflow from model-specific vector search to evaluation-ready ranking artifacts.

is to support fit-for-purpose model selection rather than confound model comparison with changes in the evaluation dataset itself.

3.5 Retrieval Evaluation Workflow

Once the vector collections are initialized, the pipeline executes the retrieval loop shown in Figure 3. Each evaluation instance begins with a synthetic query drawn from the shared benchmark set. That query is submitted to a model-specific ChromaDB collection, where the embedding model under evaluation determines the vector representation used for search. The system then returns a ranked list of retrieved chunks under a fixed cutoff, ensuring that every model is tested at the same retrieval depth. The shared evaluation settings that constrain this loop are summarized in Section 4.4 (Chroma 2026).

The ranked results are subsequently aligned with the previously constructed qrels so that each query is interpreted against the same binary relevance resource, regardless of which embedding model produced the ranking. This ordering is methodologically important because corpus segmentation, query construction, and relevance judgments are all fixed before comparative retrieval begins. In combination with collection reinitialization, this design isolates representation quality from changes in benchmark artifacts or index state, allowing observed differences to be attributed more directly to the interaction between the candidate embedding model and the corpus.

The retrieval stage produces two classes of outputs. First, it emits evaluation-ready ranking artifacts that are scored into retrieval metrics under the fixed cutoff; these per-query ranked lists are persisted for every model, which is what allows the uncertainty stage of Section 3.6 to resample the per-query metric values without repeating retrieval. Second, it records the implemented timing traces: combined corpus embedding and index-insertion timing for corpus construction, and combined query-embedding and vector-search latency for query execution. These timing by-products do not alter the retrieval protocol itself, but they allow the benchmark to preserve measured operational evidence alongside effectiveness-oriented outputs under a single controlled workflow.

3.6 Query-Level Uncertainty Estimation

A ranking of point estimates does not by itself say whether a corpus is able to separate two candidate models, and a practitioner who acts on an unresolved difference has been given a false signal. The pipeline therefore treats uncertainty as an evaluation stage rather than as an optional post-hoc analysis. During retrieval evaluation, each model's per-query ranked list is persisted alongside the aggregate metrics, so the per-query metric values that the aggregate is built from remain available. This stage resamples those values and reports the result as part of the dataset-level artifact, which is the level at which the selection decision is made.

Two kinds of interval are produced, and they answer different questions. A *marginal* interval describes how far a single model's score would move under resampling of the query set. These intervals are wide and overlap heavily, because every model evaluated on a corpus is exposed to the same queries and therefore shares that corpus's difficulty variation. A *paired* interval instead resamples the per-query differences between a model and the corpus's own leading candidate, which cancels the shared component. The paired interval is the one that discriminates, and overlap between two marginal intervals is consequently not evidence that two models perform alike. Both are reported so that the distinction remains visible to the reader rather than being resolved silently.

Resampling uses the percentile bootstrap with B resamples at a configured coverage level, and the pipeline draws each (corpus, model, metric) cell from its own independently derived random stream. This makes the reported interval for a corpus independent of how many datasets were evaluated in the same invocation, of the order in which they were processed, and of which metrics were enabled: evaluating one corpus alone reproduces exactly the numbers obtained when evaluating all of them together. The alternative, a single stream consumed in traversal order, would make every reported interval depend on unrelated parts of the run and would not be reproducible from a partial re-execution.

The same estimator is applied at the aggregate level, with two properties that must be stated because they change interpretation. It is a pooled mean over all scored queries rather than the unweighted macro mean of the per-corpus means used elsewhere in the reports, so the two aggregate figures differ slightly. It is also unstratified: resampling draws uniformly from the concatenated per-query vector and does not preserve each corpus's share of the queries, so variation in corpus composition is folded into the aggregate interval widths. These intervals are descriptive and conditional on the snapshot; because queries are generated from corpus chunks, queries are nested within a small number of datasets, documents contribute related chunks, and relevance labels are model-assigned, they should not be read as population-level inference.

3.7 Automated Reporting and Reproducibility

The final stage transforms raw benchmark outputs into structured reporting artifacts intended for comparative analysis and engineering reuse. As illustrated in Figure 4, this workflow is organized in two reporting layers whose outputs are summarized in Table 2, and both are generated from stored artifacts rather than from transient run state. At the end of retrieval evaluation the pipeline first materializes the complete evaluation

record as a machine-readable results file: the applied configuration, corpus and query statistics, qrels composition, per-model retrieval metrics, per-query rankings, timing measurements, and an environment snapshot. Every subsequent reporting product is derived from that file, which is why a report can be regenerated, or a reporting option changed, without repeating retrieval.

The first layer operates separately for each dataset or dataset subset. When the objective is to select a suitable embedding model for one available corpus, or for a representative sample drawn from that corpus, the pipeline can terminate at this stage. The generated dataset report is a self-contained document: it restates the configuration and model roster under which the corpus was evaluated, describes the corpus, query set, and relevance-judgment composition, presents the per-model retrieval metrics together with the marginal and paired query-level intervals of Section 3.6 and a pairwise difference summary, reports embedding throughput and query latency, and closes with the corpus-specific reading and its limitations. Metric-profile and timing charts are emitted alongside it. These per-dataset artifacts preserve the full evaluation trace for one corpus under the shared model roster and provide the self-contained basis for a corpus-specific shortlist within that local context.

The second layer begins only after one or more dataset-level reports have been completed. This aggregation stage is optional secondary context for settings in which the available collection is heterogeneous, for example when documents belong to different topic classes such as finance, law, or administration, or when the corpus spans multiple languages such as English, Turkish, or French. In such cases, a practitioner may evaluate heterogeneous language or domain subsets separately, generate a dataset report for each subset, and then combine the completed reports into an aggregate comparison package, as shown in Figure 4. The aggregate report is therefore derived from completed benchmark products rather than from raw retrieval outputs: it loads the stored per-dataset result files and reports cross-dataset rankings by average rank, per-dataset model performance, embedding-speed and latency comparisons, pooled query-level intervals, and a per-dataset recapitulation, so that cross-corpus behavior can be inspected without overriding the corpus-specific shortlist produced by each dataset-level report. The concrete experimental configuration and the resulting comparative evidence are presented in Sections 4 and 5, respectively.

The method is designed for reproducibility at both the artifact and workflow levels. Intermediate benchmark resources are materialized as explicit corpus, query, qrels, ranking, timing, and report artifacts; model evaluation is performed on isolated vector collections; and reporting outputs are stored as timestamped run products at both dataset and aggregate levels. The implementation of this pipeline is publicly available for inspection and reuse, which supports transparent reconstruction of the workflow described in this paper. This availability should be understood as implementation transparency and artifact-level reuse, not as an immutable archival record or a guarantee that external services and model aliases will remain unchanged.

3.8 Applying the Pipeline to a New Corpus

The workflow is distributed as a command-line tool, and the stages of Table 1 map onto its invocations. A practitioner supplies a directory of source documents for the

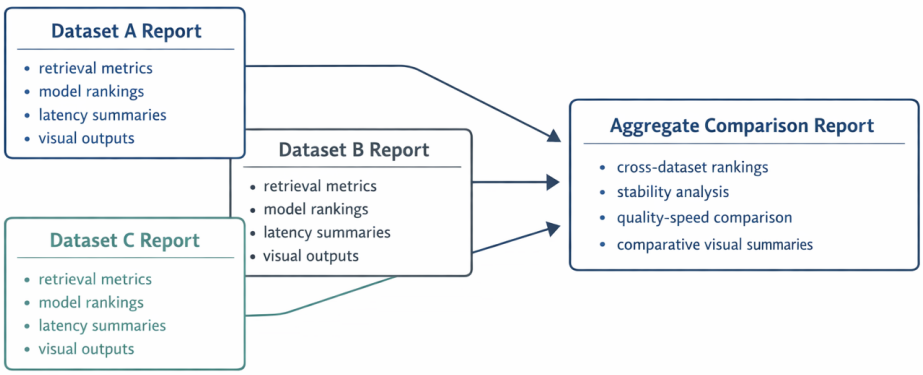


Fig. 4 Reporting flow from dataset-specific benchmark outputs to aggregate comparison artifacts.

new corpus and sets, in one configuration file, the candidate model roster, chunking parameters, query count, candidate-pool depth, retrieval cutoff, and judge settings. Nothing else is corpus-specific: dataset routing derives the corpus's working paths from its name, so a second corpus is added by creating a second directory rather than by editing code.

Table 3 lists the invocations. The two language-model stages—query generation and relevance judging—may be run synchronously for small corpora or submitted as batch jobs for larger ones; this is the only point at which the execution mode varies. A single end-to-end invocation runs the whole sequence and skips any stage whose artifact already exists, so an interrupted run is resumed by repeating the same command. Because each stage writes its artifact to disk before the next begins, a practitioner can stop after relevance construction to inspect the generated queries and judgments before committing to the evaluation stages, or re-run only the uncertainty and reporting stages after changing a reporting option—neither re-embeds the corpus nor issues further language-model calls.

4 Experiments

4.1 Experimental Snapshot

This section instantiates the generic benchmarking pipeline described in Section 3 under one selected experimental configuration whose values are held fixed for controlled comparison. The three corpora are worked demonstrations of separate corpus-specific embedding-model selection under one completed protocol. They create meaningful variation in language, domain, document structure, and scale, but they do not isolate causal language or domain effects. The objective is therefore to show how a practitioner can apply the same protocol independently to several local document collections and read each dataset-level report as the primary basis for a corpus-specific shortlist.

The experimental design follows a reusable decision-support pattern. A practitioner first defines one or more local document collections, applies the same preprocessing

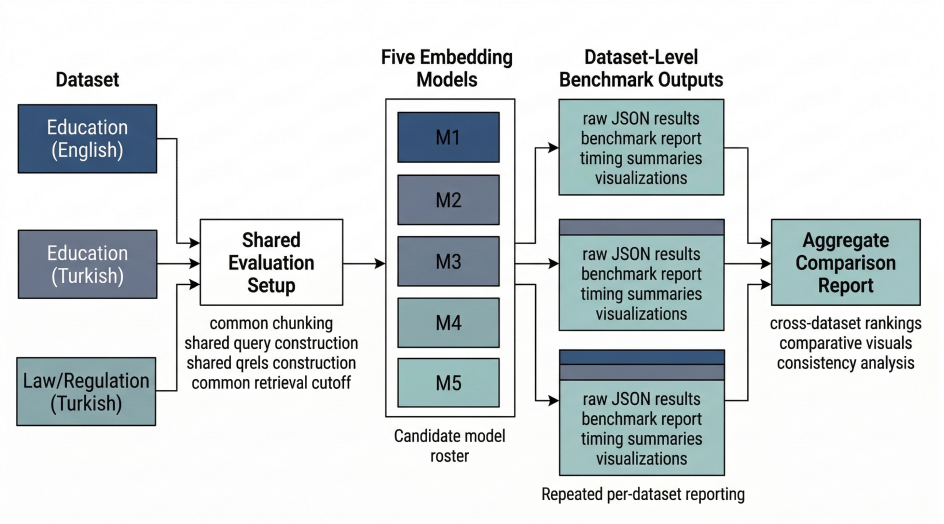


Fig. 5 Experimental design used to instantiate the proposed benchmarking pipeline in this paper.

and benchmark-construction procedure to each collection, and evaluates the same five-model roster under one shared setup. As shown in Figure 5, this process yields repeated dataset-level benchmark outputs for each corpus before any higher-level comparison is attempted. Separate reports demonstrate how different corpora may support different choices under the same roster and fixed retrieval protocol. The aggregate comparison stage is then constructed from these completed per-dataset outputs as secondary cross-corpus context rather than as an alternative evaluation branch or a command to deploy one global model.

4.2 Datasets

The benchmark covers three local datasets chosen to expose corpus variation within one controlled setup. Two corpora belong to the education domain and are provided in English and Turkish, whereas the third corpus belongs to the Turkish legal and regulatory domain. For compact reference in the paper, these datasets are denoted as `education_eng`, `education_tr`, and `law_tr`, respectively. This composition allows the same evaluation workflow to be examined across corpora that differ in language, topic, document structure, and corpus scale without treating those factors as separately identified causes. The corresponding quantitative workload summary and source-provenance boundary are reported in Table 4 and Table 5, respectively.

Table 4 summarizes the same per-dataset counts in tabular form after document ingestion, chunking, synthetic query generation, and qrels construction. The frozen experimental snapshot contains 60 source documents, 1,914 chunks, and 150 generated synthetic queries in total. The evaluator scores only positive-bearing queries, excluding queries with no positive qrels; this leaves 148 statistically scored queries, consisting of 48 for `education_eng` and 50 for each Turkish dataset. The table's positive-qrels column reports only relevance-positive labels, not all judged query–chunk pairs. Explicit

negative judgments are also stored in the qrels artifacts, as described in Section 3.3, so positive-qrels counts should not be read as total judgment counts.

Table 4 also shows an important structural characteristic of the benchmark: although `law_tr` contains only two source documents, it yields more chunks than either education dataset. The present dataset collection should therefore be read as an intentionally diverse experimental snapshot for demonstrating corpus-specific model selection under a shared protocol. In the present implementation, these source documents were processed from PDF files, but the experimental logic is framed at the document-collection level rather than at one fixed file format.

4.3 Dataset Provenance

Table 5 records the provenance boundary of the datasets used in the paper. Its columns summarize the dataset identifier used in this paper, the corresponding language-and-domain category, and the source collection from which the benchmark subset was curated. The education corpora were curated from the TED University regulations and guidelines collection (TED University 2026), whereas the law corpus was curated from the BDDK regulations listing (Bankacilik Duzenleme ve Denetleme Kurumu (BDDK) 2026). All source material was retrieved on `2026-01-01`. TED University and BDDK are the originating source publishers identified for these materials; this paper does not assert additional licenses, permissions, or redistribution rights beyond the repository artifacts and the documented source provenance. Reuse of extracted source text remains subject to the terms of the originating sources. Explicit provenance is important because the benchmark is intended as a reusable local evaluation workflow: when the same procedure is applied to a different institutional collection, domain archive, or multilingual document set, the resulting evidence should be interpreted relative to that corpus boundary.

4.4 Shared Evaluation Setup

All datasets were processed under the same evaluation configuration so that the main experimental differences arise from corpus characteristics and embedding-model behavior rather than from changes in the benchmark protocol. A central condition of the completed snapshot is its retrieval geometry: Sentence-Transformers embeddings were encoded with the default non-normalized setting and searched in ChromaDB's default squared-Euclidean (L2) space. Distances were converted to $1/(1+d)$ similarity values for reporting, but ranking remained conditional on this non-normalized squared-Euclidean configuration, referred to below as the *raw-L2* setting. No normalized-cosine or model-recommended similarity rerun is reported here, and the paper does not assume that cosine retrieval would preserve the same ordering for non-normalized vectors.

The values selected for this snapshot are demonstration settings, not tuned optima. Retrieval used the common cutoff `k=10`; chunking used a size of `1000` characters with `200` characters of overlap; 50 synthetic queries were generated per dataset; and qrels construction pooled the top-20 candidates retrieved by each of the five models into one model-independent candidate set before LLM judging (Section 3). In the pipeline

implementation, these experiment-related parameters are stored in a central configuration file, which makes the setup directly accessible and adjustable when a practitioner needs to benchmark a different corpus, alternative model roster, or modified operating constraint (Chroma 2026). They are held fixed here so that the three corpus demonstrations can be compared under one protocol. Changing any value defines a separate controlled benchmark run and may produce different scores or rankings; this paper reports no sensitivity runs for chunking, query count, pool depth, retrieval geometry, or cutoff.

For the present experimental snapshot, synthetic query generation and qrels construction both used the Gemini 2.5 Flash-Lite model. More specifically, the pipeline applies the same LLM family in two roles: first to synthesize a source-language query from each selected seed chunk, and then to act as a binary relevance judge over the retrieved candidate chunks associated with that query. The pipeline supports two operational modes for these LLM-dependent stages. A practitioner may run them through immediate request–response interaction when the benchmark is small or iterative inspection is desired, or may submit them through a batch-processing workflow that separates job submission from result retrieval and can reduce the total cost of larger benchmark runs. This dual execution design preserves the same benchmark logic while allowing the execution mode to be adapted to dataset size, budget, or turnaround constraints.

The retrieval metrics reported later use the Sentence-Transformers `InformationRetrievalEvaluator` at cutoff 10. For a scored query q with positive set R_q and binary relevance indicator $r_{q,i}$ at rank i, zero-positive queries with $|R_q| = 0$ are excluded before metric averaging. `MAP@10` averages $\mathrm{AP@10}(q) = \frac{1}{\min(|R_q|,10)} \sum_{i=1}^{10} P_q(i) r_{q,i}$, where $P_q(i)$ is precision through rank i; positives not retrieved in the top 10 therefore lower the score through the denominator. `Recall@10` is the fraction of R_q retrieved in the top 10, `Precision@10` is the fraction of the 10 returned items that are positive, and `NDCG@10` is binary-gain DCG normalized by the ideal ranking for that query at the same cutoff. `MRR@10` is the reciprocal rank of the first positive result in the top 10, or zero if none appears, and `Success@10` is the evaluator's `Accuracy@10`: one if at least one positive appears in the top 10 and zero otherwise.

The aggregate concepts used in Section 5 are derived only from existing dataset-level outputs. A dataset macro mean is an unweighted arithmetic mean over the three corpus-level metric values, so aggregate mean `MAP@10` is not a pooled average over all 148 scored queries. Average rank is the arithmetic mean of a model's rank positions by `MAP@10` within the three datasets, with lower values indicating better average placement. Weighted query latency is the scored-query-weighted mean of per-dataset query latencies, using the 48/50/50 scored-query counts in Table 4.

Table 7 lists the embedding model roster used in the experimental configuration. The table identifies each model, its embedding dimension, and its role within the benchmarked roster. This roster contains five accessible Sentence-Transformers candidates selected partly through popularity and accessibility in the Sentence Transformers Hugging Face repository (Sentence Transformers 2026b; Hugging Face 2026). These choices make the roster practical for a reproducible demonstration, but they

do not make it comprehensive. The snapshot does not include lexical retrieval, hybrid retrieval, reranking, domain-adapted embedding models, newer multilingual alternatives, or commercial embedding providers. The purpose of the present setup is to demonstrate that the proposed pipeline can evaluate a realistic open candidate set under controlled conditions; all model-selection claims are therefore conditional on this five-model roster and fixed protocol.

Throughout the paper, *English-oriented* is shorthand for the `all-MiniLM` and `all-mpnet` models, which are trained predominantly on English data, while *multilingual* denotes the `paraphrase-multilingual` models, which are trained on many languages. These labels describe the intended training regime and are used for discussion; they are not strict claims about tokenizer coverage or guaranteed per-language quality.

To preserve comparability across datasets, the experiments held constant the metric definitions and retrieval cutoff, the embedding model roster, the candidate pool size used during qrels construction, the raw-L2 retrieval geometry, the chunking parameters summarized in Table 6, and the reporting pipeline. The intentionally varying factors were corpus language, domain, corpus size, and document composition. This separation between fixed and varying factors provides a practical guideline for external reuse: a practitioner who wishes to benchmark a different corpus can keep the evaluation frame fixed while substituting a new dataset collection and, if needed, a different candidate model list.

4.5 Reproducibility Conditions

Table 8 summarizes the software, data-processing, and measurement conditions required to inspect or rerun the benchmark; library versions are pinned in the public repository. Seed-chunk sampling uses a fixed random seed, and each run records an environment snapshot (Python and library versions, device) in its raw-result JSON. The repository makes the implementation and available benchmark artifacts inspectable and reusable, but it is a live repository rather than an immutable archive, and this paper does not claim a DOI-backed or tagged archival record. External service behavior and model aliases may change outside the frozen artifacts.

The operational timing measurements are descriptive single-run, single-machine, CPU-only observations. Embedding throughput combines corpus embedding and ChromaDB index insertion, while query latency combines query embedding and vector search. These compound measurements are useful for comparing the frozen runs under one local setup, but they do not isolate model inference from vector-store behavior, report repeated-trial variability, or measure runtime memory.

4.6 Report Outputs and Utility

The reporting outputs of these experiments follow the two-layer design of Section 3.7. Each of the three corpora produced its own dataset report with the contents described there, and those three reports are the primary decision-support artifacts of the snapshot; the aggregate stage then combined them into one cross-dataset comparison, used

here as secondary context. Within the experimental section these reporting products are operational outputs of the benchmark procedure rather than findings in themselves; the substantive comparative evidence derived from them is presented in Section 5.

5 Results

This section reports the evidence produced by the proposed benchmarking pipeline under the selected experimental instantiation described in Section 4. The primary reading is corpus-specific rather than global: each corpus is read from its own dataset report before any cross-corpus summary is considered, and Section 5.1 states the resulting per-corpus selections. Those selections are conditional on the frozen five-model roster, synthetic queries, LLM judgments, raw-L2 retrieval configuration, and compound single-machine timing measurements described in Section 4. They demonstrate how to interpret one configured benchmark run; they are not default recommendations supplied by the pipeline for other corpora or parameter settings. Because the pipeline generates full benchmark reports automatically, only a selected subset of the most decision-relevant tables and figures is included here; the complete generated reports and supporting files are available through the public repository for interested readers (Karakaya 2026).

5.1 Corpus-Specific Selection Outcomes

The reported three-dataset snapshot first supports separate dataset-level readings. Table 9 summarizes the corpus-specific quality outcomes: `M1` leads the English education corpus by `MAP@10`, while `M5` leads both Turkish corpora. The timing tables in Sections 5.2–5.4 complete the practical interpretation by showing that `M1` is also the lowest-latency candidate for `education_eng`, whereas `M3` is the lower-latency compromise among the two Turkish shortlist candidates. A practitioner reading these reports for one organization could therefore select `M1` for an English education corpus, use `M5` where Turkish retrieval quality is the priority, or use `M3` where Turkish latency matters more. The different English/Turkish choices illustrate why one model should not be assumed across all corpora, while the similar Turkish shortlists show that local evaluation can also confirm stability under a shared protocol.

Because the dataset-level report is the primary selection artifact, the pipeline emits the query-level uncertainty layer for each corpus as part of that report; Table 10 collects the three. The two columns answer the different questions distinguished in Section 3.6: the marginal column is wide and overlaps heavily on all three corpora because it carries the corpus's shared query-difficulty variation, while the paired column resamples the per-query *differences* against that corpus's own `MAP@10` leader and is therefore the one that discriminates.

The per-corpus layer changes how one of the three dataset reports should be read, and confirms the other two. On `education_eng` the paired interval for `M1`−`M2` includes zero, so this corpus does not separate the two leading candidates on retrieval quality, although `M1` does separate from `M4`, `M5` and `M3`. On both Turkish corpora every paired interval against `M5` excludes zero, including the narrow gap to the runner-up `M3`, so

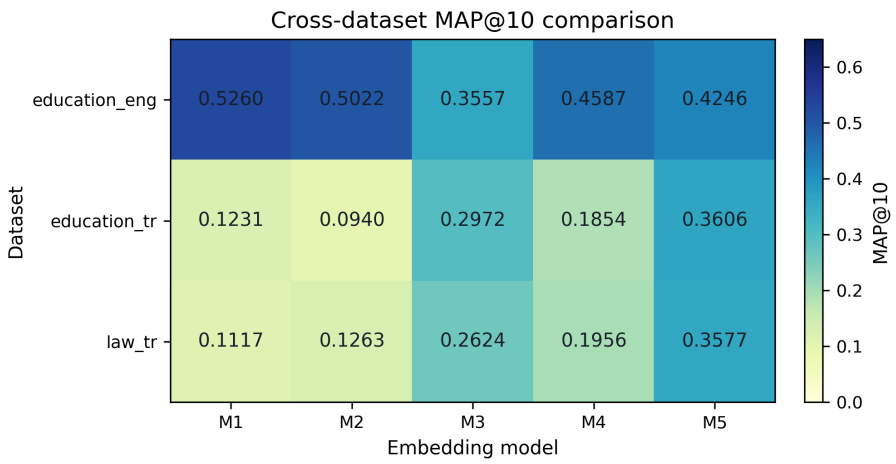


Fig. 6 Cross-dataset `MAP@10` comparison for the five benchmarked embedding models, shown as secondary context after the corpus-specific selections.

the `M5`-over-`M3` quality ordering is supported at the query level and not only as a point estimate. This is the practical function of the layer: it tells the practitioner which parts of a corpus ranking are load-bearing and which are not, and where the decision must fall back to operational cost.

Secondary aggregate context is still useful, but it should not override the dataset-level reports. Table 11 shows that `M5` has the best average rank, highest mean `MAP@10`, and two dataset wins across the fixed snapshot. Figure 6 shows the same cross-dataset `MAP@10` pattern: the English benchmark sits at a higher absolute score level, while the Turkish benchmarks are lower and led by the multilingual candidates. This aggregate view is therefore comparative context for RQ1–RQ3, not a universal deployment recommendation.

5.2 Dataset-Level Results: education_eng

The English education subset produces the strongest retrieval scores in the present paper. Table 12 shows that the three English-oriented models (`M1`, `M2`, and `M4`) lead, with `M1` reaching the highest `MAP@10` at 0.5260, `M2` close behind at 0.5022, and `M4` a little lower at 0.4587; the multilingual models `M5` (0.4246) and `M3` (0.3557) trail. Figure 7 illustrates this English-oriented ranking pattern across the full metric profile.

Table 13 completes the practical reading, and the query-level evidence in Table 10 determines how much of it rests on retrieval quality. On quality, this corpus does not separate the two leading candidates: the paired bootstrap interval for `M1`−`M2` over the 48 scored queries is +0.024 [−0.044, +0.093] and includes zero, so the 0.5260 versus 0.5022 gap is not a difference this benchmark can resolve. What does separate them is operational cost: `M1` answers a query in 13.10 ms against 42.62 ms for `M2`, and indexes the corpus at 51.3 against 7.7 chunks/s. `M1` is separated from the remaining candidates on quality as well, with paired intervals excluding zero against `M4`, `M5` and `M3`. For this English education corpus the benchmark therefore points to `M1` under the completed

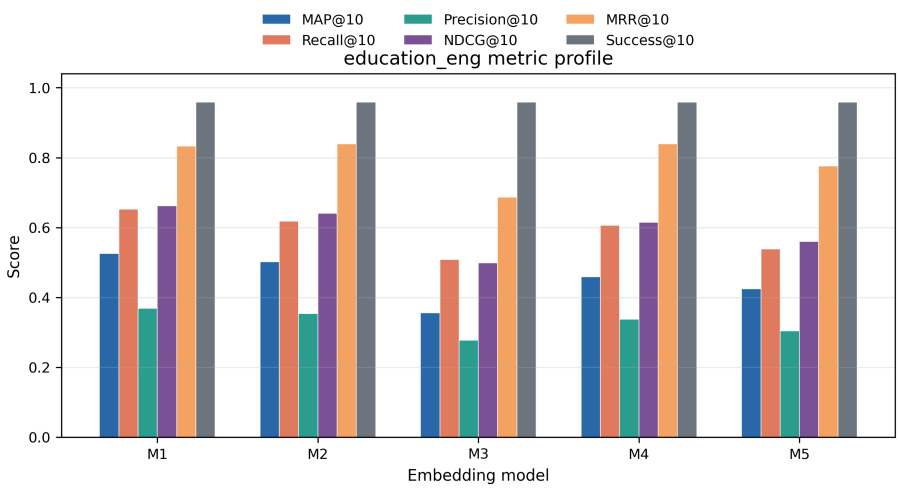


Fig. 7 Representative per-dataset metric profile generated automatically for the `education_eng` benchmark.

raw-L2 protocol—not because it is measurably more effective than `M2`, but because it is statistically indistinguishable from it at roughly one third of the latency—with `M4` as a lightweight alternative at measurably lower quality (0.4587 at 24.31 ms/query). This is the pattern the pipeline is designed to expose: where the quality evidence does not discriminate, the operational evidence decides.

5.3 Dataset-Level Results: education_tr

The Turkish education subset reverses the preferred model family. As shown in Table 14, the multilingual models `M5` and `M3` stand above the three English-oriented alternatives across all six retrieval metrics, with `M5` reaching 0.3606 `MAP@10`, `M3` reaching 0.2972, and the next English-oriented model `M4` reaching 0.1854. Figure 8 shows the same multilingual-versus-English-oriented pattern across the full metric profile.

Unlike `education_eng`, this corpus separates its leader on quality: the paired bootstrap interval for `M5`−`M3` over the 50 scored queries is +0.063 [+0.017, +0.112] and excludes zero, as do the intervals against the three remaining candidates (Table 10). The `M5` advantage is therefore load-bearing rather than a point-estimate artifact. Table 15 shows that it is obtained at different costs: `M3` remains relatively close to the lightweight models at 25.63 ms/query, whereas the higher-capacity `M5` rises to 42.72 ms/query. For Turkish education corpora, the pipeline therefore reduces the practical candidate set to `M5` and `M3`: `M5` is preferred when retrieval effectiveness is prioritized, while `M3` is the lower-latency compromise that trades a measurable quality decrement for roughly 40% lower query latency.

5.4 Dataset-Level Results: law_tr

The Turkish law/regulation subset is of similar overall difficulty to `education_tr` (mean `MAP@10` 0.2107 versus 0.2121) and preserves the same model-family ordering.

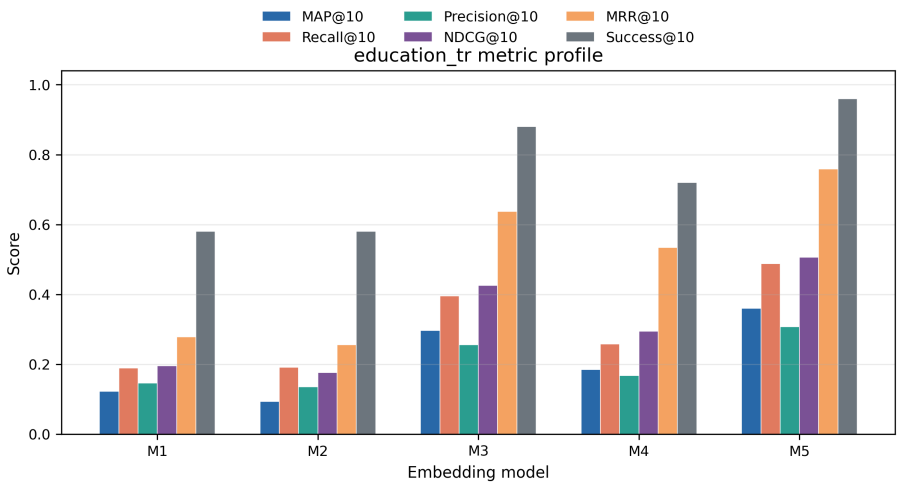


Fig. 8 Representative per-dataset metric profile generated automatically for the `education_tr` benchmark.

Table 16 shows that `M5` remains the `MAP@10`-leading candidate at 0.3577, followed by `M3` at 0.2624, while the remaining models trail. Compared with `education_tr`, per-model `MAP@10` changes only modestly—`M5` and `M3` are almost flat (within 0.035), `M2` and `M4` edge slightly up, and `M1` edges slightly down—so within Turkish the shift from education to law/regulation is associated with limited ranking change in this snapshot. Figure 9 illustrates the same multilingual-led ordering at a similar absolute level to `education_tr`.

The query-level evidence again supports the ordering: the paired interval for `M5`−`M3` is +0.095 [+0.033, +0.162] and excludes zero, as do the intervals against the other candidates (Table 10). Table 17 shows that the operational ranking remains broadly similar to the other datasets. For this legal and regulatory corpus, the benchmark therefore supports the same `M5`/`M3` shortlist as Turkish education, on the same footing: `M5` is the quality-first candidate with a separated advantage, and `M3` is the lower-latency compromise. The single-corpus-per-domain design means this observed stability should be confirmed on the specific target domain before any broader transfer assumption is made.

5.5 Cross-Language Shortlist Stability: The Education Corpora

Under the shared evaluation setup in Section 4.4, the English and Turkish education corpora provide a within-snapshot comparison of whether model behavior differs between the two language-associated cases. Because the comparison rests on a single corpus per language, language is confounded with corpus identity, source selection, and qrels composition; the comparison is therefore a bounded case observation rather than an isolated causal estimate. The two corpora are drawn from the same institutional regulation set and are not disjoint: four documents appear in both in translation, namely the long-term academic leave regulation, the quality-commission directive, and two

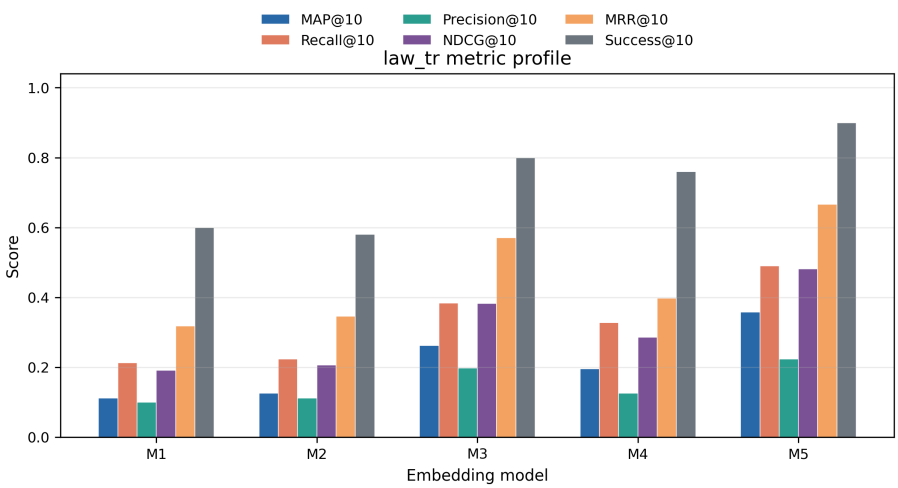


Fig. 9 Representative per-dataset metric profile generated automatically for the `law_tr` benchmark.

Erasmus+ mobility calls. They are not parallel collections, however. The remaining documents have no counterpart and differ in emphasis, with `education_eng` weighted toward student-facing academic regulation such as admission criteria, examination rules, internships, and scholarships, and `education_tr` weighted toward institutional administration such as budgeting, procurement, fixed-asset inventory, and signature authority. Language therefore co-varies with document composition, which is a further reason to read this subsection as a comparison of two corpus reports produced under one protocol rather than as an estimate of a language effect. Table 18 shows that the English results sit in a relatively narrow high band, with all five models between 0.3557 and 0.5260 in `MAP@10`, whereas the Turkish results are lower and more spread by model family.

The English-oriented models (`M1`, `M2`, and `M4`) all decline substantially on `education_tr`: `M1` drops by 0.4029, `M2` by 0.4082, and `M4` by 0.2733. By contrast, the multilingual models are more stable across the two education corpora, each dropping only about 0.06 (`M3` by 0.0585 and `M5` by 0.0640). The evidence therefore suggests a language-associated model-family pattern within this snapshot rather than a universal rule, and it also shows that `M1` leads `education_eng`, so an English-oriented model can still be the strong candidate when the corpus language and retrieval priority align with that model family.

The practical implication is narrow but useful. For local retrieval deployment, language should be treated as a corpus characteristic worth benchmarking separately: the `MAP@10`-leading candidate for an English corpus is not necessarily the quality-first candidate for a Turkish corpus, even when both corpora are evaluated under the same workflow. That is the main value of this comparison for the paper's fit-for-purpose claim.

5.6 Cross-Domain Shortlist Stability: The Turkish Corpora

Under the shared evaluation setup in Section 4.4, Table 19 compares what changes when the corpus stays Turkish but the domain-associated case shifts from education to law/regulation. As with the language-associated comparison, this rests on one corpus per domain, so domain is confounded with corpus structure—notably `law_tr`'s two long regulatory documents—and the comparison is a case observation rather than a general causal estimate.

The comparison shows that, in this snapshot, the education-to-law/regulation difference within Turkish is small and mixed. The multilingual pair remains the strongest on both Turkish corpora, with `M5` and `M3` holding the top two positions in each dataset, and their scores are essentially flat across the comparison (`M5` −0.0029, `M3` −0.0348); the English-oriented models move only slightly and in mixed directions (`M2` +0.0323, `M4` +0.0102, `M1` −0.0114). The practical reading is that local evaluation can confirm stability as well as difference: these two Turkish corpora support the same leading shortlist, but the design has only one corpus per domain, so a practitioner facing a different Turkish domain should still benchmark it directly rather than assume transfer.

5.7 Quality-Speed Tradeoff

Table 20 combines secondary aggregate quality with compound timing evidence, and Figure 10 makes the same tradeoff visible as a two-dimensional ranking. The points form three useful regimes. `M5` occupies the quality-leading end of the aggregate view with the highest mean `MAP@10` at 0.3810, but it does so at a weighted latency of 42.94 ms/query and a throughput range of 13.3–13.9 chunks/s. `M1` anchors the speed-leading end with the lowest weighted latency at 16.28 ms/query and the highest throughput range of 42.3–51.3 chunks/s, but its mean `MAP@10` is 0.2536 because it performs poorly on the Turkish corpora. `M2` is the weakest aggregate choice in this snapshot because it combines the lowest mean `MAP@10` (0.2408) with the highest weighted latency (51.66 ms/query) and the slowest throughput range (3.5–7.7 chunks/s).

Within this roster, the multilingual models `M3` and `M5` provide the strongest aggregate retrieval quality, while low latency is concentrated in the lightweight MiniLM models (`M1`, and to a lesser extent `M3` and `M4`); speed is not a pure family pattern, because the English-oriented MPNet model `M2` is the slowest model overall. This pattern should not be treated as universal: `M1` leads `education_eng`, showing that an English-oriented model can still be an effective choice when the corpus and deployment priority align.

The middle of the plot contains `M3` and `M4`, the most plausible compromise candidates when neither extreme is acceptable. `M3` is the stronger of the two on aggregate retrieval quality, with mean `MAP@10` of 0.3051 versus 0.2799, at a similar latency (25.05 ms/query versus 26.91), and the two share an average rank of 3.00. On this evidence, no single model leads all corpus-specific and operational priorities jointly: the preferred embedding model depends on whether the practitioner values retrieval effectiveness, indexing cost, or interactive response time more strongly for the corpus at hand. The per-corpus selections this implies are those already given in Section 5.1.

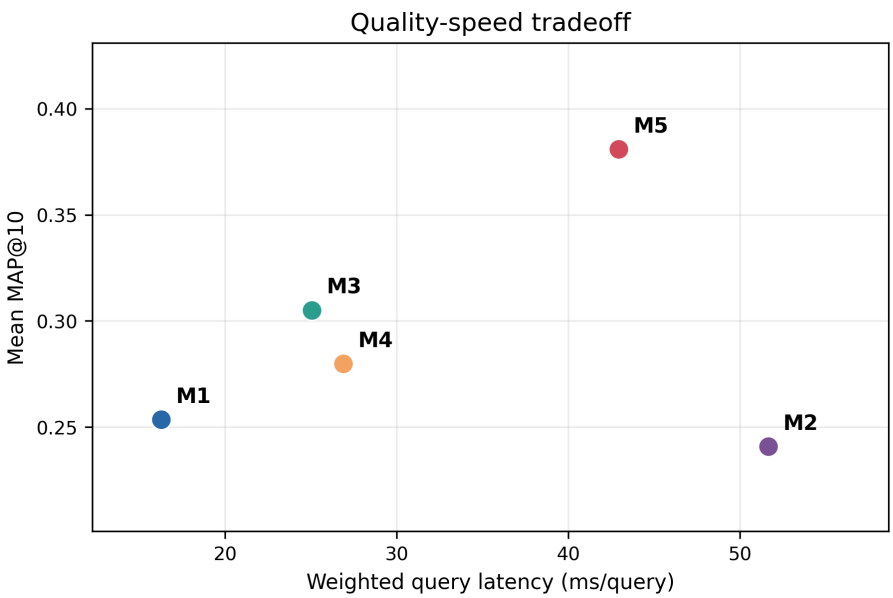


Fig. 10 Quality-speed tradeoff scatter plot for the benchmarked embedding models under the fixed raw-L2, single-machine snapshot.

5.8 Aggregate Query-Level Uncertainty

The benchmark is intended not only to rank embedding models but to show how much query-level variation surrounds the observed aggregate `MAP@10` values. The experiment generated 150 queries, but the evaluator scores only positive-bearing queries; after excluding two zero-positive `education_eng` queries, the statistical summaries use 148 scored queries, as defined in Table 4. Following the uncertainty stage described in Section 3.6, we report percentile bootstrap 95% confidence intervals for each model's aggregate `MAP@10` (B = 10,000 resamples) and paired bootstrap intervals for the difference between `M5` and each other model in Table 21, with the per-model intervals visualized in Figure 11. For the reasons given in Section 3.6, these intervals are descriptive and should not be read as population-level inference.

Read descriptively, `M5` has the highest aggregate `MAP@10`, and the paired intervals for `M5`−model differences are positive for the other four candidates. The intervals also keep the snapshot's limits visible: the per-model intervals are wide and overlap, and the ordering is not uniform across conditions. On `education_eng`, the English-oriented models lead and `M5` trails the `MAP@10`-leading candidate `M1`, whereas on both Turkish corpora `M5` leads. The two properties of this aggregate estimator set out in Section 3.6 apply here concretely: it is a pooled mean over the 148 scored queries, so it differs slightly from the macro-mean aggregates in Table 20, and it is unstratified over the 48/50/50 split across corpora.

The corresponding per-corpus layer, which is what the dataset reports carry and what the selections in Section 5.1 rest on, is given in Table 10.

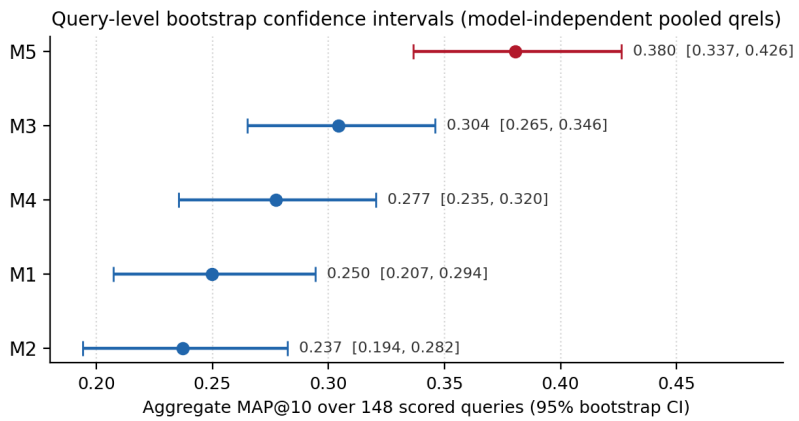


Fig. 11 Forest plot of aggregate `MAP@10` over the 148 scored queries with 95% percentile bootstrap confidence intervals (model-independent pooled qrels). The intervals are descriptive and conditional on the fixed snapshot.

5.9 Qualitative Examples and Error Analysis

To make the benchmark task concrete, consider the `education_eng` query `q1_for_English_chunk24`, *"What are the requirements for students to pass each level and advance in the ELS program?"*, generated from seed chunk `English_chunk24` of the English-language-school regulations. Thirty-seven pooled candidates were judged for this query and six were labeled relevant. One of them, `English_chunk12`, describes how a student is "placed in the next level of the one they have completed successfully," directly answering the question, whereas the judged-non-relevant near-miss `Admission_chunk2` lists GCE A-Level entry grades, which is topically adjacent (levels and passing grades) but concerns admission rather than in-program progression. The Turkish examples are analogous: `education_tr` query `q1_for_Bütçe_chunk2` asks about the university's budget and accounting principles, and `law_tr` query `q1_for_Sermaye_chunk238` asks about the reorganization of departments and representative offices in capital-market institutions. All identifiers are those used in the released `queries.jsonl` and `qrels.tsv` files. These examples illustrate the kind of corpus-grounded synthetic queries and near-miss negative judgments produced by the pipeline; they are not a human validation study or a frequency analysis of model-specific errors.

Three observations temper and contextualize the results. First, in this snapshot each query's seed chunk is judged like any other candidate rather than force-labeled relevant; this removes the guaranteed-positive bias of a forced-seed protocol and is reflected in a high but no-longer-saturated `Success@10` on `education_eng` (about 0.96 across models rather than 1.00). Two consequences of that choice are worth reporting explicitly, because together they bound how incomplete the candidate pool is. The seed chunk is nevertheless judged relevant for 47, 44, and 44 of the 50 queries in `education_eng`, `education_tr`, and `law_tr` respectively, so it remains a positive for about 90% of queries even without forcing. Conversely, for 9 of the 150 queries

(0, 4, and 5 respectively) no evaluated model retrieved the seed chunk within its top-20, so the chunk that generated the query never entered the candidate pool and was never judged at all. Because the seed chunk is known to be on-point by construction, this is a direct measurement of pool incompleteness within the evaluated roster rather than a general caveat, and it bounds that incompleteness at a small fraction of queries under model-independent pooling (Zobel 1998; Buckley and Voorhees 2004). Second, the model-independent pooled qrels store explicit negative judgments alongside positives—1,805, 3,024, and 3,515 non-relevant labels for `education_eng`, `education_tr`, and `law_tr` respectively—so non-relevant decisions are inspectable rather than discarded. Third, the examples help explain why the paper treats the quantitative rankings as preliminary decision-support evidence: without human audit or judge-agreement data, the examples can illustrate the retrieval task but cannot independently verify query realism, label reliability, or model-specific failure rates.

6 Discussion

6.1 Practitioner implications

The main practical implication of the results is that a practitioner should read the benchmark output at the corpus level before considering any aggregate summary. The three dataset reports show that one organization can reasonably make different embedding-model choices for different document collections under the same evaluation roster and protocol. In the frozen snapshot, `M1` is the preferred candidate for `education_eng` on operational grounds—it is not separated from `M2` on retrieval quality by the per-corpus paired intervals, but answers queries in roughly one third of the time; `M5` is the quality-first candidate for `education_tr` and `law_tr`, where its advantage over the runner-up is separated at the query level; and `M3` provides the lower-latency Turkish compromise when response time matters more than that measured increment in retrieval effectiveness. The per-corpus uncertainty layer is what makes this distinction visible: it identifies which corpus rankings are decided by quality and which fall through to cost. These are examples of how to interpret the generated reports, not permanent prescriptions for future corpora.

In practical reuse, the practitioner first configures a benchmark for the corpus, candidate roster, and application requirements, then holds those choices constant while comparing the candidates and inspecting the dataset-level report. If the initial report motivates a different model roster or alternative retrieval, chunking, query, or evaluation settings, the pipeline can be rerun under that revised configuration as a separate sensitivity experiment. The present paper demonstrates the first decision-support cycle under one selected setup; it does not measure the effect of changing those settings.

This reading also clarifies the role of the aggregate report. The aggregate view shows that `M5` has the strongest cross-dataset quality summary in the present snapshot, but that summary does not override the dataset-level evidence recorded above. The intended workflow is therefore preliminary corpus-specific decision support: it narrows the candidate set under an explicit protocol and exposes quality–speed tradeoffs, while leaving final deployment choice to the practitioner's corpus, tolerance for synthetic evidence, and operational constraints.

6.2 Language- and domain-aware corpus partitioning

Language and domain remain important because they help determine when a local collection should be partitioned and benchmarked separately. The English and Turkish education corpora produce different model-family rankings in this snapshot, whereas the Turkish education and Turkish law/regulation corpora preserve the same leading shortlist. The lesson is not that either language or domain is an isolated cause. Each comparison involves one corpus per condition, and corpus identity includes source selection, document structure, chunk distribution, generated queries, and qrels composition. The safer implication is that observed difference and observed stability should both be measured rather than assumed.

For practitioners, this supports a partition-first interpretation of the pipeline. If a local repository contains multiple languages, regulatory genres, institutional policy documents, or structurally different document types, the workflow can be run on each materially distinct subset. Separate reports then show whether the same candidate remains suitable or whether the shortlist changes. In the present paper, language-associated variation appears in the education comparison, while domain-associated stability appears in the Turkish comparison; both outcomes are useful because they prevent an unsupported transfer from one corpus report to another.

6.3 Relation to prior benchmark workflows

The contribution of this paper is an integrated local benchmark workflow rather than a new embedding algorithm, retrieval metric, or general-purpose leaderboard. Prior RAG and retrieval-evaluation work emphasizes the importance of retrieval quality, synthetic queries, LLM judging, benchmark governance, and reproducibility controls (Lewis et al. 2020; Zhu et al. 2026; Thakur et al. 2025; Abdallah et al. 2025; Miyaji et al. 2025; Wang et al. 2025; Friel et al. 2024). The pipeline connects those concerns in the single practitioner-facing sequence of Table 1. As set out in Section 2.5, this places the workflow upstream of answer-quality evaluation frameworks and alongside, rather than in place of, the metric and dataset libraries on which it depends.

This positioning is intentionally modest. The paper does not claim that the five evaluated models exhaust the current embedding landscape or that the selected protocol is preferable to all alternatives. Instead, it demonstrates how a practitioner can turn a local document collection into reusable benchmark artifacts and compare an accessible candidate roster under one controlled setup. The value lies in making the comparison repeatable, inspectable, and reportable for each corpus, especially when manual trials would otherwise mix preprocessing choices, query sets, judgment policies, retrieval geometry, and reporting criteria.

6.4 Limitations and transfer limits

Several limits constrain how far the reported rankings should be transferred. The benchmark uses synthetic queries and same-family LLM judging: Gemini 2.5 Flash-Lite generates the queries and also judges candidate relevance. The paper includes illustrative examples and stored positive and negative labels, but it includes no human validation, no human–LLM agreement analysis, and no independent audit of query

realism or label reliability. The experiment generated 150 queries, but statistical scoring used 148 positive-bearing queries after zero-positive exclusion. The qrels are also roster-conditional because they are built from the union of candidates retrieved by the five evaluated models; relevant chunks missed by every pooled system may remain unjudged (Zobel 1998; Buckley and Voorhees 2004), and the qrels should not be treated as complete ground truth for arbitrary future systems.

The retrieval and protocol choices are similarly bounded. All rankings are conditional on default non-normalized Sentence-Transformers embeddings searched with ChromaDB squared-Euclidean (L2) distance; no normalized-cosine or model-recommended similarity rerun is reported. The evaluated roster contains five Sentence-Transformers candidates and excludes lexical retrieval, hybrid retrieval, reranking, domain-adapted models, newer multilingual models, and commercial alternatives. The selected chunk size, overlap, 50-query generation target, top-20 pool depth, and `k=10` cutoff are demonstration settings rather than tuned optima. Although the pipeline permits alternative configurations, functional configurability is not evidence that the reported rankings are robust to those alternatives; no sensitivity study is included.

Operational and reproducibility claims are also descriptive. The timing evidence combines corpus embedding with index insertion and combines query embedding with vector search; it comes from a single-run, single-machine, CPU-only snapshot and does not measure repeated-trial variability or runtime memory. The repository makes implementation and benchmark artifacts inspectable, but a live repository is not the same as an immutable archived release with a DOI, fixed service behavior, and frozen model aliases. These limits mean the pipeline output should be used as preliminary corpus-specific decision support, not as a deployment guarantee or as evidence that the observed ordering will transfer unchanged to another corpus, model roster, retrieval geometry, or hardware environment.

7 Conclusion

This paper presented a reusable, configuration-driven local benchmarking pipeline for corpus-specific embedding-model selection in RAG retrieval. The workflow processes each document collection separately through ingestion, chunking, synthetic query generation, pooled qrels construction, multi-model retrieval evaluation, timing capture, and automated reporting. Its central research position is deliberately bounded: practitioners should not assume that one embedding model will be optimal for every corpus under their control. The three-corpus snapshot demonstrates how the workflow tests that assumption; it does not prove that a universal best model cannot exist.

For RQ1, the fixed snapshot shows that the configurable pipeline can produce inspectable, corpus-specific decision-support evidence. Separate reports for `education_eng`, `education_tr`, and `law_tr` contain comparable retrieval metrics, timing summaries, query-level uncertainty, and preserved artifacts under the same five-model protocol. These reports support corpus-specific screening rather than a single cross-corpus selection rule.

For RQ2, the two education cases show language-associated ranking differences: English-oriented candidates lead `education_eng`, whereas multilingual candidates lead

`education_tr`. This comparison does not establish a causal language effect because each language condition is represented by one corpus. For RQ3, the two Turkish cases instead show domain-associated stability: `M5` remains the quality-first candidate and `M3` the lower-latency alternative for both `education_tr` and `law_tr`. This stability does not show that domain is irrelevant; together, RQ2 and RQ3 demonstrate why both shortlist change and shortlist stability must be measured through separate corpus runs.

For RQ4, the reports show that retrieval quality and operational priorities can produce different corpus-specific selections. They combine retrieval metrics with compound corpus embedding/index-insertion throughput, compound query-embedding/vector-search latency, and uncertainty intervals. What the uncertainty layer adds is visible in the demonstrated scenario: on `education_eng` the quality advantage of the leading candidate over its runner-up lies within the paired bootstrap interval, so the selection falls to latency, whereas on both Turkish corpora the corresponding advantage is separated and the quality evidence decides. The aggregate performance of `M5` remains secondary comparative context and does not override these corpus-level readings.

The current limitations define focused directions for strengthening the reliability and breadth of corpus-specific recommendations. Limited validation of synthetic queries and LLM-generated qrels calls for human audit and judge calibration; the fixed experimental configuration calls for controlled sensitivity studies through the existing configuration interface; bounded corpus and model coverage calls for additional matched corpora that examine language- and domain-associated differences or stability, broader embedding-model rosters, and contextual retrieval baselines; incomplete operational evidence calls for repeated measurements of timing variability, peak memory, and model size on declared hardware; and mutable reproducibility artifacts call for immutable archived releases containing configurations, prompts, artifacts, checksums, and environment metadata.

The practical takeaway is that no universally best embedding model should be assumed: practitioners should evaluate each materially distinct corpus separately, treat aggregate results as secondary context, and select according to corpus-specific evidence and project priorities.

Declarations

Funding. This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Competing interests. The author declares that there are no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Ethics approval and consent to participate. Not applicable. This study did not involve human participants, animal subjects, or personal data. The benchmark corpora consist of publicly available institutional regulations and legislation retrieved from the sources listed in Table 5.

Consent for publication. Not applicable.

Data availability. The benchmark artifacts reported in this paper are available in the project repository (Karakaya 2026). The released artifacts comprise the source documents, chunked corpora, generated synthetic queries, LLM-judged qrels including explicit negative labels, per-model ranking files, raw-result JSON, and the generated dataset-level and aggregate reports for all three corpora. Persisted ChromaDB vector indexes are excluded from the release and can be rebuilt from the chunked corpora with the pipeline's indexing stage. The source documents are public institutional regulations obtained from the providers listed in Table 5 (TED University 2026; Bankacilik Duzenleme ve Denetleme Kurumu (BDDK) 2026); the reproducibility conditions under which the reported snapshot was produced are summarized in Table 8.

Materials availability. Not applicable.

Code availability. The pipeline implementation is available in the project repository (Karakaya 2026), together with the configuration and stage sequence used to produce the reported snapshot.

Author contributions. M.K. is the sole author and was responsible for conceptualization, methodology, software, investigation, writing of the original draft, and writing—review and editing.

References

Abdallah A, Abdalla M, Piryani B, et al (2025) Rerankarena: A unified platform for evaluating retrieval, reranking and rag with human and llm feedback. In: Proceedings of the 34th ACM International Conference on Information and Knowledge Management, pp 6593–6597

Abimbola JO, Kuaban GS, Ajayi SA (2026) Open-source embedding models: A comprehensive survey of techniques, benchmarks, and applications. IEEE Access 14:41284–41302. https://doi.org/10.1109/ACCESS.2026.3670100

Ali SJ, Naganathan V, Bork D (2024) Establishing traceability between natural language requirements and software artifacts by combining RAG and LLMs. In: International Conference on Conceptual Modeling, Springer, pp 295–314, https://doi.org/10.1007/978-3-031-75872-0_16

Antal M (2025) Evaluation of embedding models for hungarian question-answer retrieval on domain-specific and public benchmarks. Infocommunications journal 17(4):2–11. https://doi.org/10.36244/icj.2025.4.1, URL http://dx.doi.org/10.36244/icj.2025.4.1

Bagheri Nezhad S, Agrawal A (2025) Cross: Analyzing the trade-offs in long-context cross-lingual retrieval. In: ICLR 2025 Workshop on Foundation Models in the Wild

Bankacilik Duzenleme ve Denetleme Kurumu (BDDK) (2026) Mevzuat listesi. https://www.bddk.org.tr/Mevzuat/Liste/49, accessed: 2026-01-01

Bassani E, Pasi G (2021) Semantic query labeling through synthetic query generation. In: Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval. ACM, SIGIR '21, pp 2278–2282, https://doi.org/10.1145/3404835.3463071, URL http://dx.doi.org/10.1145/3404835.3463071

Baysan MS, Gungor T (2025) Tr-mteb: A comprehensive benchmark and embedding model suite for turkish sentence representations. In: Findings of the Association for Computational Linguistics: EMNLP 2025. Association for Computational Linguistics, pp 8867–8887, https://doi.org/10.18653/v1/2025.findings-emnlp.471, URL http://dx.doi.org/10.18653/v1/2025.findings-emnlp.471

Bellard B, Liu XC, Lohani R, et al (2025) Development and evaluation of an agentic rag with embedded llm-as-a judge for heart failure management: A pilot study. In: 2025 IEEE International Conference on Big Data (BigData), pp 7770–7774, https://doi.org/10.1109/BigData66926.2025.11402164

Buckley C, Voorhees EM (2004) Retrieval evaluation with incomplete information. In: Proceedings of the 27th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '04). ACM, pp 25–32, https://doi.org/10.1145/1008992.1009000

Chase H (2022) LangChain. https://github.com/langchain-ai/langchain, software, first released 2022-10-17; accessed 2026-01-01

Chroma (2026) Chromadb documentation. https://docs.trychroma.com, accessed: 2026-01-01

Cohen D, Burg L, Pykhnivskyi S, et al (2025) Wixqa: A multi-dataset benchmark for enterprise retrieval-augmented generation. arXiv preprint arXiv:250508643

Cummings A, Zhang X, Olaniran M, et al (2025) Rag pipeline for domain specific applications: A case study in disseminating dementia care practices. In: Proceedings of the ACM/IEEE International Conference on Connected Health: Applications, Systems and Engineering Technologies. Association for Computing Machinery, New York, NY, USA, CHASE '25, pp 139–143, https://doi.org/10.1145/3721201.3721396, URL https://doi.org/10.1145/3721201.3721396

Dvořáčková L (2025) Analyzing word embeddings and their impact on semantic similarity: Through extreme simulated conditions to real dataset characteristics. Neural Computing and Applications 37(19):13765–13793. https://doi.org/10.1007/s00521-025-11231-4

Enevoldsen K, Chung I, Kerboua I, et al (2025) MMTEB: Massive multilingual text embedding benchmark. In: The Thirteenth International Conference on Learning Representations (ICLR), arXiv:2502.13595

Es S, James J, Espinosa Anke L, et al (2024) RAGAs: Automated evaluation of retrieval augmented generation. In: Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations. Association for Computational Linguistics, St. Julians, Malta, pp 150–158, https://doi.org/10.18653/v1/2024.eacl-demo.16

Friel R, Belyi M, Sanyal A (2024) RAGbench: Explainable benchmark for retrieval-augmented generation systems. arXiv preprint arXiv:240711005

Guţu BM, Popescu N (2025) From dataset to model: A romanian–english corpus and fine-tuned cross-lingual embeddings for text and tabular retrieval. Applied Sciences 15(22):12219. https://doi.org/10.3390/app152212219, URL http://dx.doi.org/10.3390/app152212219

Hugging Face (2026) Sentence-transformers model hub. https://huggingface.co/sentence-transformers, accessed: 2026-01-01

Iana A, Glavaš G, Paulheim H (2026) Multilinguality in mind: Advancing cross-lingual news recommendation with a multilingual dataset. ACM Transactions on Recommender Systems 4(3). https://doi.org/10.1145/3777415, URL https://doi.org/10.1145/3777415

Iscan C, Ozara MF, Akbulut A (2024) Enhancing rag pipeline performance with translation-based embedding strategies for non-english documents. In: 2024 Innovations in Intelligent Systems and Applications Conference (ASYU). IEEE, pp 1–6, https://doi.org/10.1109/ASYU62119.2024.10756977, URL http://dx.doi.org/10.1109/asyu62119.2024.10756977

Kalantari M, Feghhi M, Mozayani N (2024) Benchmarking embedding models for persian-language semantic information retrieval. In: 2024 15th International Conference on Information and Knowledge Technology (IKT), pp 53–58, https://doi.org/10.1109/IKT65497.2024.10892801

Karakaya M (2025) Local benchmarking of sentence embedding models for retrieval-augmented generation. In: Proceedings of the International Conference on Computer Science and Intelligence Systems (ICCSIS), IIER, Madrid, Spain, pp 1–6, URL https://www.iraj.in/journal/journal_file/journal_pdf/12-1119-17744404741-7.pdf, proceedings version published in the International Journal of Advances in Electronics and Computer Science (IJAECS) 12(9), ISSN 2394-2835

Karakaya M (2026) Ragretrievalbench repository and benchmark datasets. https://github.com/kmkarakaya/RagRetrievalBench.git, accessed: 2026-01-01

Khan MZ, Ge Y, Ullah U, et al (2025) Telcogpt: A hybrid embedding approach for telecom-specific q&a and code retrieval. In: IEEE INFOCOM 2025 - IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS), pp 1–6, https://doi.org/10.1109/INFOCOMWKSHPS65812.2025.11152931

Kunuku MT, Dehbozorgi N (2025) Multimodal quiz generation via rag with llm-as-judge evaluation. In: 2025 IEEE Frontiers in Education Conference (FIE). IEEE, pp 1–7, https://doi.org/10.1109/FIE63693.2025.11328244, URL http://dx.doi.org/10.1109/fie63693.2025.11328244

Lewis P, Perez E, Piktus A, et al (2020) Retrieval-augmented generation for knowledge-intensive NLP tasks. In: Advances in Neural Information Processing Systems, pp 9459–9474

Lin J, Ma X, Lin SC, et al (2021) Pyserini: A python toolkit for reproducible information retrieval research with sparse and dense representations. In: Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '21). Association for Computing Machinery, pp 2356–2362, https://doi.org/10.1145/3404835.3463238

Liu J (2022) LlamaIndex. https://github.com/run-llama/llama_index, software, first released 2022-11-02; accessed 2026-01-01

Luo H, Luo X, Zhao W, et al (2026) Domain-specific retrieval-augmented generation with adaptive embedding and knowledge distillation-based re-ranking. Processes 14(1):99. https://doi.org/10.3390/pr14010099

MacAvaney S, Yates A, Feldman S, et al (2021) Simplified data wrangling with ir_datasets. In: Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '21). Association for Computing Machinery, pp 2429–2436, https://doi.org/10.1145/3404835.3463254

MacAvaney S, Macdonald C, Ounis I (2022) Streamlining evaluation with ir-measures. In: Advances in Information Retrieval: 44th European Conference on IR Research (ECIR 2022), Proceedings, Part II, Lecture Notes in Computer Science, vol 13186. Springer, Cham, pp 305–310, https://doi.org/10.1007/978-3-030-99739-7_38

Mekonnen KA, Alemneh YW, de Rijke M (2025) Optimized text embedding models and benchmarks for amharic passage retrieval. In: Findings of the Association for Computational Linguistics: ACL 2025, pp 10428–10445

Mikkelsen Y (2026) Clinical context variables collectively rival model choice in embedding-based retrieval: Multi-corpus benchmark study. JMIR Medical Informatics 14:e94241. https://doi.org/10.2196/94241, URL https://doi.org/10.2196/94241

Miyaji R, Moulin R, Moncao S, et al (2025) Evaluating rag-based qa systems: A comparative analysis of llm as a judge, traditional metrics, and human alignment. In: Simposio Brasileiro de Tecnologia da Informacao e da Linguagem Humana (STIL), SBC, pp 247–258

Muennighoff N, Tazi N, Magne L, et al (2023) MTEB: Massive text embedding benchmark. In: Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics. Association for Computational Linguistics, Dubrovnik, Croatia, pp 2014–2037, https://doi.org/10.18653/v1/2023.eacl-main.148

Reimers N, Gurevych I (2019) Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In: Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP). Association for Computational Linguistics, Hong Kong, China, pp 3982–3992, https://doi.org/10.18653/v1/D19-1410

Saad-Falcon J, Khattab O, Potts C, et al (2024) ARES: An automated evaluation framework for retrieval-augmented generation systems. In: Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers). Association for Computational Linguistics, Mexico City, Mexico, pp 338–354, https://doi.org/10.18653/v1/2024.naacl-long.20

Sajja R, Sermet Y, Demir I (2025) Domain-specific embedding models for hydrology and environmental sciences: enhancing semantic retrieval and question answering. Water Science & Technology 92(9):1328–1342. https://doi.org/10.2166/wst.2025.156

Sentence Transformers (2026a) Pretrained models documentation. https://www.sbert.net/docs/sentence_transformer/pretrained_models.html, accessed: 2026-01-01

Sentence Transformers (2026b) Sentence transformers repository at hugging face. https://huggingface.co/sentence-transformers/models?sort=downloads, accessed: 2026-01-01

Szabó BN, Gyöngyössy NM, Dalos A (2025) Applied domain adaptation of llm-based document embeddings for engineering knowledge retrieval. In: Proceedings of the Intelligent Robotics FAIR 2025. Association for Computing Machinery, New York, NY, USA, IntRob '25, pp 29–37, https://doi.org/10.1145/3759355.3759360, URL https://doi.org/10.1145/3759355.3759360

Taemkaeo C, Saetia C, Chalothorn T, et al (2026) Enhancing hr support in a thai organization with llm-based question answering. In: 2026 18th International Conference on Knowledge and Smart Technology (KST), pp 1–6, https://doi.org/10.1109/KST67832.2026.11431871

TED University (2026) Regulations and guidelines for education. https://www.tedu.edu.tr/en/regulations-and-guidelines-education, accessed: 2026-01-01

Thakur N, Reimers N, Rücklé A, et al (2021) BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models. In: Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track (Round 2), URL https://openreview.net/forum?id=wCu6T5xFjeJ

Thakur N, Pradeep R, Upadhyay S, et al (2025) Assessing support for the TREC 2024 RAG track: A large-scale comparative study of LLM and human evaluations. In: Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval. ACM, pp 2759–2763, https://doi.org/10.1145/3726302.3730165

Upadhyay S, Pradeep R, Thakur N, et al (2024) UMBRELA: UMbrela is the (open-source reproduction of the) Bing RELevance Assessor. arXiv preprint arXiv:240606519

Voorhees EM (2002) The philosophy of information retrieval evaluation. In: Evaluation of Cross-Language Information Retrieval Systems, Lecture Notes in Computer Science, vol 2406. Springer, p 355–370, https://doi.org/10.1007/3-540-45691-0_34

Wang B, Zhao D, Tallent NR, et al (2025) On the reproducibility limitations of RAG systems. arXiv preprint arXiv:250918869

Wang Y, Birman KP (2025) Diagnosing and resolving cloud platform instability with multi-modal rag llms. In: Proceedings of the 5th Workshop on Machine Learning and Systems. Association for Computing Machinery, New York, NY, USA, EuroMLSys '25, pp 139–147, https://doi.org/10.1145/3721146.3721958, URL https://doi.org/10.1145/3721146.3721958

Wen C, Zhu A, Long R, et al (2026) Calibjudge: Calibrated llm-as-a-judge for multilingual rag with uncertainty-aware scoring. In: Proceedings of the 2026 International Conference on Artificial Intelligence and Control. Association for Computing Machinery, New York, NY, USA, CAIC '26, pp 661–667, https://doi.org/10.1145/3807246.3807349, URL https://doi.org/10.1145/3807246.3807349

Xu X, Weytjens H, Zhang D, et al (2025) RAGops: Operating and managing retrieval-augmented generation pipelines. arXiv preprint arXiv:250603401

Xu Y (2025) An end-to-end benchmarking framework for retrieval-augmented generation systems. Master's thesis, University of Illinois Urbana-Champaign, URL https://hdl.handle.net/2142/132602

Yoon J, Arik SO (2025) Embedding-converter: A unified framework for cross-model embedding transformation. In: Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pp 25464–25482

Yozkan E, Supriyanto I (2025) How reliable is semantic search in industrial computing domain? a statistical evaluation pipeline. In: 2025 International Conference on Artificial Intelligence, Computer, Data Sciences and Applications (ACDSA), pp 1–6, https://doi.org/10.1109/ACDSA65407.2025.11166412

Yunianto I, Permanasari AE, Widyawan (2020) Domain-specific contextualized embedding: A systematic literature review. In: 2020 12th International Conference on Information Technology and Electrical Engineering (ICITEE). IEEE, pp 162–167, https://doi.org/10.1109/ICITEE49829.2020.9271752, URL https://doi.org/10.1109/ICITEE49829.2020.9271752

Zhu Y, Yuan H, Wang S, et al (2026) Large language models for information retrieval: A survey. ACM Transactions on Information Systems 44(1). https://doi.org/10.1145/3748304

Zobel J (1998) How reliable are the results of large-scale information retrieval experiments? In: Proceedings of the 21st Annual International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '98). ACM, pp 307–314, https://doi.org/10.1145/290941.291014

Table 1 Stage sequence and intermediate artifacts of the proposed embedding benchmarking workflow.

Stage	Input	Process	Output
Dataset routing	Local dataset selection	Bind one corpus to isolated working paths so downstream artifacts remain separated across corpora	Dataset-specific run context
Document ingestion	Routed document collection	Parse source documents and normalize extracted text for downstream processing	Textual corpus
Chunking	Textual corpus	Segment documents into overlapping units suitable for embedding and retrieval	Chunked corpus
Query generation	Chunked corpus	Sample seed chunks and generate natural-language queries in the source language of the seed content	Synthetic query set
Relevance construction	Query set and chunked corpus	Retrieve a bounded candidate pool for each query and assign binary relevance labels through LLM-assisted judgment	Qrels resource
Multi-model indexing	Chunked corpus and selected embedding models	Build a separate vector collection for each embedding model so retrieval runs remain isolated and reproducible	Per-model vector collections
Retrieval evaluation	Query set, qrels, and per-model collections	Execute the same fixed-depth retrieval workflow for every model over the same benchmark artifacts	Ranking artifacts and timing traces
Uncertainty estimation	Per-query ranking artifacts and qrels	Resample the per-query metric values to obtain marginal intervals per model and paired intervals against the corpus's leading candidate, with each cell drawn from an independently seeded stream	Per-corpus confidence intervals
Reporting	Dataset-level evaluation artifacts	Produce a dataset-level report for corpus-specific selection and, when multiple reports exist, aggregate comparison artifacts	Benchmark reports and visual summaries

Table 2 Reporting outputs produced by the proposed benchmarking pipeline.

Output level	Artifacts	Purpose
Dataset level	Benchmark report, ranking summaries, timing summaries, per-query ranking files, per-corpus confidence intervals, and dataset-specific visualizations	Support a corpus-specific shortlist for one corpus, one subset, or one representative sample under the shared model set, and record which parts of that shortlist the corpus actually resolves
Aggregate level	Cross-dataset comparison report, pooled confidence intervals, and comparative visual summaries	Consolidate heterogeneous dataset-level outputs into optional context for cross-corpus or cross-subset inspection

Table 3 Pipeline stages and their command-line invocations. Every invocation takes `--dataset <name>`, which performs the dataset-routing stage.

Stage	Invocation	LLM
Document ingestion and chunking	`--prepare-corpus`	no
Query generation	`--generate-queries`	yes
Relevance construction	`--generate-qrels`	yes
Multi-model indexing	`--initialize`	no
Retrieval evaluation, uncertainty estimation, and dataset report	`--evaluate`	no
Uncertainty estimation alone	`--bootstrap`	no
Dataset report alone	`--regenerate-report`	no
Aggregate report	`--generate-aggregate-report`	no
Whole per-corpus sequence	`--all`, or `--build-all` for batch mode	yes

Table 4 Dataset statistics summary for the frozen experimental snapshot. *Judged pairs* counts every query–chunk pair drawn from the model-independent candidate pool and submitted to the LLM judge; the positive qrels are the subset of those pairs labeled relevant, and the remainder are stored as explicit negative labels.

Dataset	Source documents	Chunks	Generated queries	Scored queries	Judged pairs	Positive qrels
education_eng	29	572	50	48	2,117	312
education_tr	29	619	50	50	3,378	354
law_tr	2	723	50	50	3,801	286
Total	60	1,914	150	148	9,296	952

Table 5 Provenance and document composition of the benchmark datasets used in this paper. The two education corpora share four documents in translation but are not parallel collections; see Section 5.5.

Dataset	Language and domain	Document type	Source
education_eng	English education	University regulations and guidelines, weighted toward student-facing academic rules (admission, examinations, internships, scholarships)	TED University regulations and guidelines
education_tr	Turkish education	University regulations and guidelines, weighted toward institutional administration (budgeting, procurement, fixed assets, signature authority)	TED University regulations and guidelines
law_tr	Turkish law/regulation	Banking and financial-sector regulation	BDDK regulations listing

Table 6 Parameter values selected for the reported experimental snapshot.

Parameter	Value	Role in the workflow
Chunk size	1000	Determines the maximum character length of each corpus segment before embedding
Chunk overlap	200	Preserves continuity between adjacent chunks and reduces boundary-induced information loss
Queries per dataset	50	Fixes the number of synthetic queries generated for each corpus before zero-positive exclusion during scoring
Chunks per query seed	1	Anchors each generated query to one sampled source chunk during query construction
Candidate pool size	20	Limits the number of retrieved candidates examined during relevance construction
Document batch size	32	Controls batched embedding throughput during vector collection construction
Collection reinitialization	True	Rebuilds vector collections so each benchmark run starts from a controlled index state
Shared retrieval cutoff	10	Fixes the number of ranked chunks returned per query during retrieval evaluation

Table 7 Embedding model roster used in the experimental configuration.

ID	Model	Dimension	Role in the benchmark
M1	sentence-transformers/all-MiniLM-L6-v2	384	English-oriented MiniLM; lightweight efficiency baseline
M2	sentence-transformers/all-mpnet-base-v2	768	English-oriented MPNet; higher-capacity English-oriented model
M3	sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2	384	Multilingual paraphrase MiniLM; compact cross-language model
M4	sentence-transformers/all-MiniLM-L12-v2	384	English-oriented MiniLM; deeper L12 variant
M5	sentence-transformers/paraphrase-multilingual-mpnet-base-v2	768	Multilingual paraphrase MPNet; higher-capacity cross-language model

Table 8 Reproducibility conditions for the benchmark snapshot.

Component	Setting
Language runtime	Python 3.10 (`ragbenchmark` conda environment)
Vector store	ChromaDB 1.0.7 (persistent client), default squared-Euclidean (L2) space; similarity $1/(1+d)$ for reporting
Embeddings	Sentence-Transformers 4.1.0 (PyTorch 2.7.0), default non-normalized encoding, batch size 32
Text extraction	PyMuPDF 1.26.0 page text with NFC Unicode normalization
Chunking	LangChain recursive character splitter, size 1000, overlap 200
Retrieval evaluation	Sentence-Transformers `InformationRetrievalEvaluator`, cutoff $k = 10$, zero-positive queries excluded before scoring
Query generation and judging	Gemini 2.5 Flash-Lite, temperature 0.1, in batch and interactive modes
Candidate pool	Model-independent union of the top-20 chunks retrieved by all five models, LLM-judged with stored positive and negative labels
Seed policy	Fixed random seed (42) for seed-chunk sampling, so generated queries can be reconstructed under the recorded workflow
Uncertainty layer	Pipeline stage (Section 3.6): per-query ranked lists persisted per model; percentile and paired bootstrap intervals ($B = 10{,}000$ resamples, coverage 0.95) emitted per corpus with the dataset report, and pooled without stratification for the aggregate. Each (corpus, model, metric) cell draws from a stream derived independently from the run seed (42), so intervals are reproducible from a partial re-execution
Timing scope	Single-run CPU-only local timing; query latency = query embedding + vector search; embedding throughput/time = batched corpus embedding + index insertion; runtime memory not measured
Hardware / OS / device	Environment snapshot (Python and library versions, device) recorded in each raw-result JSON; no additional CPU, RAM, thread, or storage matrix is reported
Available artifacts	Source PDFs, chunked corpora, generated queries, LLM-judged qrels, per-model ranking files, raw-result JSON, and generated dataset reports are tracked in the public repository (Karakaya 2026); persisted ChromaDB vector indexes are excluded and can be rebuilt from the chunked corpora. Repository availability is not an immutable archival guarantee.

Table 9 Corpus-specific performance and selection summary for the reported experimental snapshot.

Dataset	Corpus-specific interpretation	Top MAP@10	Mean MAP@10 (across models)
education_eng	`M1` (MiniLM-L6) leads quality and latency	0.5260	0.4534
education_tr	`M5` (multi-MPNet) quality-first; `M3` (multi-MiniLM) lower-latency compromise	0.3606	0.2121
law_tr	`M5` (multi-MPNet) quality-first; `M3` (multi-MiniLM) lower-latency compromise	0.3577	0.2107

Table 10 Per-corpus bootstrap intervals for `MAP@10` ($B = 10{,}000$ percentile resamples over each corpus's own scored queries, model-independent pooled qrels). The paired column gives leader−model differences; *Sep.* records whether that interval excludes zero. Intervals are descriptive and snapshot-conditional.

Model	`MAP@10` [95% CI]	Leader−model Δ [95% CI]	Sep.
education_eng ($n = 48$ scored queries; leader `M1`)			
M1	0.526 [0.457, 0.593]	—	—
M2	0.502 [0.431, 0.575]	+0.024 [−0.044, +0.093]	**no**
M4	0.459 [0.398, 0.522]	+0.067 [+0.020, +0.113]	yes
M5	0.425 [0.344, 0.509]	+0.101 [+0.021, +0.177]	yes
M3	0.356 [0.283, 0.432]	+0.170 [+0.093, +0.241]	yes
education_tr ($n = 50$ scored queries; leader `M5`)			
M5	0.361 [0.293, 0.430]	—	—
M3	0.297 [0.233, 0.365]	+0.063 [+0.017, +0.112]	yes
M4	0.185 [0.137, 0.237]	+0.175 [+0.089, +0.264]	yes
M1	0.123 [0.074, 0.177]	+0.238 [+0.155, +0.319]	yes
M2	0.094 [0.061, 0.130]	+0.267 [+0.195, +0.343]	yes
law_tr ($n = 50$ scored queries; leader `M5`)			
M5	0.358 [0.275, 0.445]	—	—
M3	0.262 [0.199, 0.330]	+0.095 [+0.033, +0.162]	yes
M4	0.196 [0.123, 0.280]	+0.162 [+0.079, +0.249]	yes
M2	0.126 [0.076, 0.188]	+0.231 [+0.140, +0.323]	yes
M1	0.112 [0.071, 0.157]	+0.246 [+0.167, +0.331]	yes

Table 11 Secondary cross-dataset aggregate ranking summary for the benchmarked embedding models.

Model	Avg Rank	Mean MAP@10 (across corpora)	Wins
M5 (multi-MPNet)	2.00	0.3810	2
M3 (multi-MiniLM)	3.00	0.3051	0
M4 (MiniLM-L12)	3.00	0.2799	0
M1 (MiniLM-L6)	3.33	0.2536	1
M2 (MPNet)	3.67	0.2408	0

Table 12 Retrieval metrics for the `education_eng` dataset. Model IDs follow Table 7; `M1` denotes MiniLM-L6.

Model	MAP@10	Recall@10	Precision@10	NDCG@10	MRR@10	Success@10[a]
M1	0.5260	0.6529	0.369	0.6626	0.8325	0.96
M2	0.5022	0.6180	0.354	0.6405	0.8398	0.96
M3	0.3557	0.5087	0.277	0.4990	0.6867	0.96
M4	0.4587	0.6065	0.338	0.6150	0.8392	0.96
M5	0.4246	0.5384	0.304	0.5595	0.7757	0.96

[a]The identical values round from 0.9583 (46/48 scored queries); the same two queries fail for all five candidates.

Table 13 Compound single-machine timing summary for the `education_eng` dataset.

Model	Chunks/s	Avg ms/chunk	Query latency (ms/query)
M1	51.3	19.49	13.10
M2	7.7	129.66	42.62
M3	44.1	22.68	24.81
M4	43.2	23.15	24.31
M5	13.9	72.16	41.07

Table 14 Retrieval metrics for the `education_tr` dataset. Model IDs follow Table 7; `M5` denotes multi-MPNet and `M3` denotes multi-MiniLM.

Model	MAP@10	Recall@10	Precision@10	NDCG@10	MRR@10	Success@10
M1	0.1231	0.1897	0.146	0.1956	0.2788	0.58
M2	0.0940	0.1913	0.136	0.1766	0.2561	0.58
M3	0.2972	0.3953	0.256	0.4259	0.6369	0.88
M4	0.1854	0.2579	0.168	0.2951	0.5345	0.72
M5	0.3606	0.4876	0.308	0.5067	0.7590	0.96

Table 15 Compound single-machine timing summary for the `education_tr` dataset.

Model	Chunks/s	Avg ms/chunk	Query latency (ms/query)
M1	43.5	23.00	16.87
M2	3.6	278.16	49.86
M3	41.3	24.19	25.63
M4	41.9	23.84	28.53
M5	13.3	75.33	42.72

Table 16 Retrieval metrics for the `law_tr` dataset. Model IDs follow Table 7; `M5` denotes multi-MPNet and `M3` denotes multi-MiniLM.

Model	MAP@10	Recall@10	Precision@10	NDCG@10	MRR@10	Success@10
M1	0.1117	0.2125	0.100	0.1915	0.3185	0.60
M2	0.1263	0.2235	0.112	0.2065	0.3457	0.58
M3	0.2624	0.3836	0.198	0.3825	0.5707	0.80
M4	0.1956	0.3278	0.126	0.2860	0.3980	0.76
M5	0.3577	0.4903	0.224	0.4815	0.6659	0.90

Table 17 Compound single-machine timing summary for the `law_tr` dataset.

Model	Chunks/s	Avg ms/chunk	Query latency (ms/query)
M1	42.3	23.66	18.74
M2	3.5	285.80	62.14
M3	41.4	24.15	24.71
M4	42.1	23.73	27.78
M5	13.5	74.24	44.95

Table 18 Language-associated `MAP@10` comparison within the education-domain corpora.

Model	education_eng MAP@10	education_tr MAP@10	Δ (eng $\rightarrow$ tr)
M1	0.5260	0.1231	-0.4029
M2	0.5022	0.0940	-0.4082
M3	0.3557	0.2972	-0.0585
M4	0.4587	0.1854	-0.2733
M5	0.4246	0.3606	-0.0640

Table 19 Domain-associated `MAP@10` comparison within the Turkish corpora.

Model	Family	education_tr MAP@10	law_tr MAP@10	Δ (law − education)
M1	English-oriented	0.1231	0.1117	−0.0114
M2	English-oriented	0.0940	0.1263	+0.0323
M3	Multilingual	0.2972	0.2624	−0.0348
M4	English-oriented	0.1854	0.1956	+0.0102
M5	Multilingual	0.3606	0.3577	−0.0029

Table 20 Secondary aggregate quality-speed summary for the benchmarked embedding models. Timing columns are compound single-machine measurements.

Model	Mean MAP@10	Avg Rank	Weighted latency (ms/query)	Throughput range (chunks/s)
M1	0.2536	3.33	16.28	42.3–51.3
M2	0.2408	3.67	51.66	3.5–7.7
M3	0.3051	3.00	25.05	41.3–44.1
M4	0.2799	3.00	26.91	41.9–43.2
M5	0.3810	2.00	42.94	13.3–13.9

Table 21 Descriptive query-level bootstrap intervals for aggregate `MAP@10` over the 148 scored queries (model-independent pooled qrels, $B = 10{,}000$ resamples). The paired column gives `M5`−model differences with 95% bootstrap intervals.

Model	Mean `MAP@10` [95% CI]	`M5`−model Δ [95% CI]
M5	0.380 [0.337, 0.426]	—
M3	0.304 [0.265, 0.346]	+0.076 [+0.045, +0.108]
M4	0.277 [0.235, 0.320]	+0.103 [+0.055, +0.153]
M1	0.250 [0.207, 0.294]	+0.130 [+0.078, +0.183]
M2	0.237 [0.194, 0.282]	+0.143 [+0.087, +0.200]